

UNIVERSIDADE FEDERAL DE ALFENAS

**BRUNA HELENA ANTONIALLI GOMES
LEONARDO HENRIQUE OLIVEIRA**

**UNICAR:
UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE
FEDERAL DE ALFENAS**

**Poços de Caldas/MG
2025**

**BRUNA HELENA ANTONIALLI GOMES
LEONARDO HENRIQUE OLIVEIRA**

**UNICAR:
UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE
FEDERAL DE ALFENAS**

Trabalho de Conclusão de Curso apresentado como parte dos requisitos para obtenção do título de Bacharelado Interdisciplinar de Ciência e Tecnologia pela Universidade Federal de Alfenas. Área de concentração: Tecnologia.

Orientador: Prof. Dr. Luiz Felipe Ramos Turci

**Poços de Caldas/MG
2025**

Sistema de Bibliotecas da Universidade Federal de Alfenas Biblioteca Campus
Poços de Caldas

Antonialli Gomes, Bruna Helena.

UNICAR : UM APLICATIVO DE CARONAS COMPARTILHADAS
PARA A UNIVERSIDADE FEDERAL DE ALFENAS / Bruna Helena
Antonialli Gomes,
Leonardo Henrique Oliveira. - Poços de Caldas, MG, 2025.
45 f. : il.

Orientador(a): Luiz Felipe Ramos Turci.

Trabalho de Conclusão de Curso (Graduação em Bacharelado
Interdisciplinar em Ciência e Tecnologia) - Universidade Federal de
Alfenas,
Poços de Caldas, MG, 2025.
Bibliografia.

1. Caronas compartilhadas. 2. Mobilidade urbana. 3. Economia
colaborativa. I. Oliveira, Leonardo Henrique. II. Ramos Turci, Luiz
Felipe, orient. III. Título.

Ficha gerada automaticamente com os dados fornecidos pelo autor.

**BRUNA HELENA ANTONIALI GOMES
LEONARDO HENRIQUE OLIVEIRA**

**UNICAR:
UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE FEDERAL DE ALFENAS**

O Presidente da banca examinadora abaixo assina a aprovação da Monografia apresentada como parte dos requisitos para a obtenção do título de Bacharel em Ciência e Tecnologia pela Universidade Federal de Alfenas. Área de concentração: Tecnologia

Aprovada em: 11 de Junho de 2025.

Prof. Dr. Luiz Felipe Ramos Turci
Presidente da Banca Examinadora
Instituição: Universidade Federal de Alfenas

Profa. Dra. Osvaldo Adilson de Carvalho Junior
Instituição: Universidade Federal de Alfenas

Profa. Dra. Teresa Cristina Monteiro Martins
Instituição: Universidade Federal de Alfenas



Documento assinado eletronicamente por **Luiz Felipe Ramos Turci, Professor(a) do Magistério Superior**, em 11/06/2025, às 17:43, conforme horário oficial de Brasília, com fundamento no art. 6º, § 1º, do [Decreto nº 8.539, de 8 de outubro de 2015](#).



A autenticidade deste documento pode ser conferida no site https://sei.unifal-mg.edu.br/sei/controlador_externo.php?acao=documento_conferir&id_orgao_acesso_externo=0, informando o código verificador **1536774** e o código CRC **B163F979**.

**BRUNA HELENA ANTONIALI GOMES
LEONARDO HENRIQUE OLIVEIRA**

**UNICAR:
UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE
FEDERAL DE ALFENAS**

O Presidente da banca examinadora abaixo assina a aprovação do Trabalho de Conclusão de Curso apresentado como parte dos requisitos para obtenção do título de Bacharel Interdisciplinar de Ciência e Tecnologia pela Universidade Federal de Alfenas. Área de concentração: Tecnologia.

Aprovada em: 11 de Junho de 2025

Prof. Dr. Osvaldo Adilson de Carvalho Junior
Universidade Federal de Alfenas - MG

Assinatura:

Prof. Dr. Teresa Cristina Monteiro Martins
Universidade Federal de Alfenas – MG

Assinatura:

Dedicamos este trabalho ao nosso orientador, Luiz Felipe Ramos Turci, pela orientação cuidadosa, pela paciência e pelo apoio constante ao longo de toda a nossa jornada. Sua dedicação, comprometimento e incentivo foram essenciais para que este trabalho se concretizasse. Agradecemos por ter acreditado em nós, mesmo quando enfrentamos desafios e incertezas. Esta conquista também é sua.

AGRADECIMENTOS

Agradecemos, em primeiro lugar, a Deus, por ter nos concedido força, sabedoria e saúde durante toda essa caminhada.

Agradecemos especialmente ao nosso orientador, cuja escuta atenta, domínio acadêmico e incentivo constante foram essenciais durante todo o desenvolvimento deste trabalho. Sua disposição em nos orientar, mesmo diante de prazos e desafios, fez toda a diferença para que pudéssemos alcançar nossos objetivos com segurança e clareza.

Aos nossos familiares, pelo amor incondicional, compreensão e incentivo nos momentos de dificuldade. Sem o apoio de vocês, nada disso seria possível.

Aos amigos que estiveram ao nosso lado durante o curso, pelas conversas, risadas, colaborações e pelo apoio emocional nos momentos mais desafiadores.

A todos os professores e professoras que contribuíram com a nossa formação acadêmica ao longo da graduação, deixando marcas importantes em nosso percurso.

Por fim, a todos que, de alguma forma, contribuíram para a realização deste trabalho — nossa sincera gratidão.

"Nunca considere o estudo como uma obrigação, mas como uma oportunidade de compreender a beleza libertadora do saber, para seu próprio prazer e benefício da sociedade."
— Albert Einstein

(EINSTEIN, 2001, p. 52)

RESUMO

O trabalho aborda o desenvolvimento de um aplicativo de caronas voltado para a comunidade acadêmica da Universidade Federal de Alfenas (UNIFAL-MG), campus Poços de Caldas. O objetivo é oferecer uma solução acessível, segura e colaborativa para os desafios de mobilidade enfrentados por estudantes, professores e funcionários. Utilizando tecnologias como MIT App Inventor e Firebase, o aplicativo busca incentivar a economia colaborativa, promovendo o compartilhamento de veículos e reduzindo custos individuais e impactos ambientais.

Palavras-chave: Caronas compartilhadas. Mobilidade urbana. Economia colaborativa.

ABSTRACT

The study focuses on the development of a ridesharing application for the academic community of the Federal University of Alfenas (UNIFAL-MG), Poços de Caldas campus. The objective is to provide an accessible, secure, and collaborative solution to the mobility challenges faced by students, professors, and staff. By leveraging technologies such as MIT App Inventor and Firebase, the application aims to promote collaborative economy practices, encouraging vehicle sharing while reducing individual costs and environmental impacts.

Keywords: Ridesharing. Urban mobility. Collaborative economy.

Sumário

1 INTRODUÇÃO	17
1.1 OBJETIVOS	18
1.1.1 Objetivo geral	18
1.1.2 Objetivos específicos.....	18
2 REVISÃO BIBLIOGRÁFICA.....	18
2.1 MOBILIDADE URBANA E COMPARTILHAMENTO DE CARONAS	18
2.2 ECONOMIA COLABORATIVA E APLICATIVOS DE MOBILIDADE	19
2.3 TECNOLOGIAS DE DESENVOLVIMENTO PARA APLICATIVOS DE CARONA	20
3. METODOLOGIA	21
4. DESENVOLVIMENTO.....	22
4.1 TELA DE LOGIN E REGISTRO (SCREEN 1)	23
4.1.1 Implementação das Funcionalidades da Screen1	26
4.2 INTERFACE DE USUÁRIO (SCREEN 4)	33
4.2.1 Implementação das Funcionalidades as Screen4	34
4.3 INTERFACE DE SOLICITAÇÃO DE CARONA (SCREEN 2)	35
4.4 INTERFACE DE SELEÇÃO DE PONTO DE EMBARQUE E AVISO AO MOTORISTA (SCREEN5)	
42	
4.4.1 Implementação das Funcionalidades na Screen5.....	43
4.5 INTERFACE DE OFERTA DE CARONA (SCREEN3).....	46
4.5.1 Implementação das Funcionalidades as Screen3	47
4.6 JORNADA DO USUÁRIO	52
	52
6. CONCLUSÃO	55
7. TRABALHOS FUTUROS.....	55
8. CONSIDERAÇÕES FINAIS	56
9- REFERÊNCIAS	56
10- APÊNDICE – FIGURAS AMPLIADAS DO APLICATIVO UNICAR	59

LISTA DE FIGURAS

Figura 1 - Tela Inicial : tela de login e cadastro de novos usuários.	24
Figura 2 - Tela com campos de nome, telefone e botão para cadastro de veículo, exibidos após o clique no botão "Registrar".	25
Figura 3 - Tela com campos de cadastro do veículo e definição de pontos de parada, exibidos após o clique em "Cadastrar carro".	26
Figura 4 - Bloco de inicialização da Screen 1.....	27
Figura 5 - Blocos que definem variáveis globais e URLs do Firebase para registro e login de usuários.	27
Figura 6 - Códigos para processo de registro de novo usuário.	28
Figura 7 - Códigos para processo de autenticação do login.	28
Figura 8 - Códigos para executar a ação de login ou registro.	29
Figura 9 - Blocos que validam o login via Firebase e ajustam a interface conforme o perfil do usuário.	30
Figura 10 - Blocos que salvam os dados do usuário no Firebase em formato de dicionário.	31
Figura 11- Blocos que controlam a atualização de ofertas no Firebase com base na data atual, apagando ou armazenando valores conforme a comparação.	31
Figura 12 - Blocos que tornam visíveis os componentes da interface ao clicar no botão "Cadastrar Carro".	32
Figura 13 - Blocos que adicionam um novo ponto de parada à lista e atualizam a interface ao clicar no botão "Adicionar Parada".	32
Figura 14 - Blocos que verificam a autenticação do usuário ao clicar em "Concluir" e redirecionam para a tela apropriada ou exibem um alerta em caso de erro.	33
Figura 15 - Tela do aplicativo com o título "Funcionalidades" e três botões: "Oferecer", "Solicitar" e "Encerrar", representando as principais ações disponíveis ao usuário.	34
Figura 16 - Bloco que define a ação do botão "Oferecer", redirecionando o usuário para a tela Screen3 ao ser clicado.	34
Figura 17 - Bloco que define a ação do botão "Solicitar", redirecionando o usuário para a tela Screen2 ao ser clicado.	35
Figura 18- Bloco que define a ação do botão "Encerrar", finalizando a aplicação ao ser clicado.	35
Figura 19 - Tela "Solicitação de carona" do aplicativo, exibindo um botão "Voltar" e uma lista para seleção de ofertas com informações sobre horário, pontos de parada, carro, cor e placa.	36
Figura 20 - Blocos de inicialização de variáveis globais utilizadas para controle e armazenamento de ofertas e vagas no aplicativo.	37
Figura 21 - Bloco que define a ação do botão "Voltar", fechando a tela atual ao ser clicado.	37
Figura 22 - Blocos responsáveis por obter e processar as ofertas ao iniciar a tela Screen2, verificando dados no Firebase e chamando funções conforme a disponibilidade de ofertas.	38
Figura 23 - Bloco da função "ler_OFERTAS" que acessa o Firebase para obter os dados de uma oferta específica com base no contador global.	39
Figura 24 - Blocos que processam dados do FirebaseDB2 para carregar ofertas, vagas e IDs, atualizando listas e controlando a exibição com base no contador.	40
Figura 25 - Bloco da função "carregar_ofertas" que exibe as ofertas no componente de lista usando os dados armazenados na variável global.	40

Figura 26 - Blocos que exibem um diálogo de confirmação ao usuário após a seleção de uma oferta na lista, mostrando a escolha e opções para confirmar ou cancelar. .	41
Figura 27 - Blocos que confirmam a escolha de uma oferta, salvam os dados no TinyDB e redirecionam ou exibem mensagem conforme a resposta do usuário.	41
Figura 28 - Blocos que atualizam os dados de uma oferta no Firebase após a seleção, ajustando o número de vagas e armazenando as informações com base no item escolhido.	42
Figura 29 - Tela de seleção do ponto de parada para embarque.	43
Figura 30 - Blocos que inicializam variáveis globais e, ao abrir a Screen5, obtêm dados do Firebase com base no identificador do ofertador armazenado no TinyDB.	44
Figura 31 - Blocos que processam dados recebidos do Firebase para montar uma mensagem personalizada.	45
Figura 32 - Blocos que atualizam a mensagem global com o ponto selecionado em uma lista suspensa e acionam o envio da mensagem via WhatsApp.	45
Figura 33 - Blocos que abrem uma conversa no WhatsApp com uma mensagem personalizada e ocultam ou exibem componentes da interface após o envio.	46
Figura 34 - Tela "Ofertamento de carona" do aplicativo, com campos para inserir horário, número de vagas, sentido da viagem e botões para cadastrar a oferta e voltar.	47
Figura 35 - Blocos de inicialização de variáveis globais usadas no controle e armazenamento de ofertas e dados de cadastro.	48
Figura 36 - Bloco que define a ação do botão "Voltar", redirecionando o usuário para a tela Screen4 ao ser clicado.	48
Figura 37 - Blocos que inicializam a tela Screen3 recuperando o ID do usuário no TinyDB e os dados de ofertas no Firebase.	49
Figura 38 - Blocos que, ao clicar no botão "Cadastrar oferta", chamam a função para buscar os dados do usuário no Firebase utilizando o ID armazenado.	49
Figura 39 - Blocos que, ao receberem dados do FirebaseDB2, armazenam os valores na variável global e chamam a função de cadastro de oferta.	50
Figura 40 - Blocos da função "montar_oferta" que compõem os dados de uma nova oferta com base nas informações inseridas pelo usuário e nos dados do cadastro, adicionando-os à lista de ofertas.	51
Figura 41 - Blocos das funções "cadastrar_oferta" e "gravar_ofertas", responsáveis por montar os dados da carona, armazená-los no Firebase e atualizar o número total de ofertas.	52
Figura 42 - UNICAR: UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE FEDERAL DE ALFENAS	53

LISTA DE ABREVIATURAS E SIGLAS

UNIFAL-MG	Universidade Federal de Alfenas
JSON	JavaScript Object Notation
MIT	Massachusetts Institute of Technology
HTTP	Hypertext Transfer Protocol
URL	Uniform Resource Locator
API	Application Programming Interface

1 INTRODUÇÃO

O crescimento do número de estudantes em instituições de ensino superior no Brasil tem trazido à tona desafios relacionados à mobilidade urbana. A necessidade de deslocamento diário de um grande contingente de alunos, professores e funcionários pressiona os sistemas de transporte público, que, em muitas regiões, não consegue atender de maneira eficiente e segura a essa demanda. Além disso, alternativas como táxis e serviços de transporte por aplicativo frequentemente se mostram financeiramente inviáveis para a maioria dos estudantes, criando um cenário de limitação de opções de transporte.

Diante desse contexto, torna-se imperativo buscar soluções que possam proporcionar aos membros da comunidade acadêmica meios de transporte que sejam acessíveis, seguros e confiáveis. Uma das alternativas que emergem com força é a prática de caronas compartilhadas, ou *carpooling*, que, além de reduzir os custos individuais de deslocamento, contribui para a diminuição do impacto ambiental gerado pelo excesso de veículos em circulação.

Diversas instituições de ensino superior no Brasil e no exterior já adotaram iniciativas voltadas ao compartilhamento de caronas como uma solução para os desafios de mobilidade urbana. Na Universidade Federal do Rio de Janeiro (UFRJ), foi desenvolvido o aplicativo Caronaê, criado por estudantes para facilitar o compartilhamento de veículos entre membros da comunidade acadêmica. Esse sistema chegou a atender entre 70 mil e 90 mil pessoas diariamente antes de ser temporariamente suspenso devido à pandemia de COVID-19 (Exame 2016).

De acordo com uma publicação no Fuvestibular (2015), ex-alunos da Universidade de São Paulo (USP) criaram o aplicativo Daily Ride, que busca oferecer trajetos mais econômicos e confortáveis para estudantes por meio de caronas compartilhadas. Além dessas iniciativas nacionais, projetos internacionais, como o *Zimride*, popular em universidades norte-americanas, conectam estudantes e professores com caronas baseadas em trajetos compartilhados. Esses exemplos evidenciam o potencial de plataformas digitais para resolver os problemas de mobilidade nas comunidades acadêmicas. (VCU NEWS, 2010; MIT NEWS, 2011; USU TODAY, 2015)

O campus da Universidade Federal de Alfenas (UNIFAL-MG) em Poços de Caldas enfrenta desafios significativos de mobilidade devido à sua localização distante dos principais centros urbanos da cidade. Esta situação é agravada pelo fato de haver apenas uma linha de ônibus que serve o campus, com horários limitados de uma vez por hora, e a ausência de benefícios como o meio-passe, tornando o transporte público uma opção menos viável para muitos estudantes, professores e funcionários. Este contexto cria uma necessidade crítica por alternativas de transporte mais eficientes e acessíveis para a comunidade acadêmica.

Este trabalho propõe o desenvolvimento de um aplicativo de caronas especificamente voltado para a comunidade acadêmica da UNIFAL-MG. O objetivo é criar uma plataforma digital que permita aos usuários – estudantes, professores e funcionários – oferecerem e buscarem caronas de maneira fácil e segura, por um custo inferior ao dos serviços de transporte tradicionais. A proposta busca incentivar a economia colaborativa dentro da comunidade acadêmica, promovendo o uso compartilhado de veículos e, conseqüentemente, otimizando os recursos disponíveis.

Para a construção do aplicativo, optou-se pela utilização do MIT App Inventor, uma ferramenta de desenvolvimento *no-code* que permite a criação de aplicativos Android de forma intuitiva e acessível. Essa plataforma foi escolhida por sua facilidade

de uso, que permite prototipagem e implementações rápidas, mesmo por desenvolvedores com pouca experiência em programação, garantindo que o desenvolvimento se mantenha ágil e eficiente.

O Firebase será empregado como banco de dados principal, fornecendo suporte robusto para autenticação de usuários, armazenamento em tempo real e hospedagem. A escolha dessas tecnologias baseou-se em sua compatibilidade, flexibilidade e no suporte abrangente que oferecem aos desenvolvedores, aspectos que serão cruciais para a implementação e a manutenção eficaz do sistema.

Este trabalho, portanto, não só pretende oferecer uma solução prática para os problemas de mobilidade urbana enfrentados pela comunidade da UNIFAL-MG, como também promover uma mudança de comportamento, estimulando o uso mais racional e coletivo dos recursos de transporte.

1.1 OBJETIVOS

Nosso trabalho tem como objetivo analisar e propor soluções viáveis para melhorar a locomoção das pessoas que precisam se deslocar diariamente até a UNIFAL, buscando alternativas eficientes, acessíveis e sustentáveis para minimizar os desafios enfrentados por esses indivíduos.

1.1.1 Objetivo geral

Desenvolver um aplicativo de caronas compartilhadas para a comunidade acadêmica da UNIFAL-MG, promovendo mobilidade acessível, segura e sustentável.

1.1.2 Objetivos específicos

- Identificar as necessidades de mobilidade dos membros da comunidade acadêmica da UNIFAL-MG.
- Projetar uma interface de usuário intuitiva e funcional utilizando o MIT App Inventor.
- Implementar funcionalidades de autenticação, armazenamento de dados e gerenciamento de caronas com o Firebase.
- Apresentar um protótipo funcional do aplicativo, destacando sua estrutura lógica, fluxos e funcionalidades implementadas.

2 REVISÃO BIBLIOGRÁFICA

2.1 MOBILIDADE URBANA E COMPARTILHAMENTO DE CARONAS

A mobilidade urbana representa um dos principais desafios das cidades contemporâneas, intensificado pelo uso crescente de veículos particulares, o que acarreta congestionamentos, poluição atmosférica e elevado consumo energético. Nesse contexto, o compartilhamento de caronas, conhecido como *carpooling*, surge como uma alternativa viável para amenizar tais problemas, ao promover uma mobilidade mais eficiente e sustentável.

Além dos impactos ambientais positivos, o *carpooling* também contribui para a saúde pública. A redução da frota circulante melhora a qualidade do ar, o que pode

diminuir a incidência de doenças respiratórias e cardiovasculares associadas à exposição à poluição. Simultaneamente, a menor quantidade de veículos em circulação tende a reduzir a ocorrência de acidentes de trânsito, promovendo maior segurança viária (INTERNATIONAL TRANSPORT FORUM, 2024)

Embora o conceito de *carpooling* não seja recente, ele ganhou destaque com o avanço das tecnologias digitais. Conforme apontado por Szymanek et al. (2022), essa prática é capaz de diminuir significativamente o número de veículos nas vias, mitigando congestionamentos e reduzindo a emissão de gases poluentes, com impactos diretos na melhoria da qualidade do ar urbano.

Para além dos benefícios ambientais e de saúde, o *carpooling* estimula a socialização e a cooperação dentro das comunidades. Segundo Ferguson (1997), plataformas digitais possibilitam a conexão entre motoristas e passageiros, fortalecendo vínculos sociais e promovendo uma cultura de colaboração comunitária. O êxito dessa prática depende de diversos fatores, como aceitação social, segurança e confiabilidade das plataformas utilizadas. A confiança dos usuários é um elemento essencial, e, conforme Ferguson (1997), a implementação de recursos como verificação de identidade e sistemas de avaliação é fundamental para garantir um ambiente seguro e transparente.

As políticas públicas também desempenham um papel estratégico na disseminação do *carpooling*. A adoção de medidas como a criação de faixas exclusivas para veículos de alta ocupação, benefícios fiscais aos motoristas participantes e a integração com o transporte público são apontadas como iniciativas eficazes para ampliar a adesão ao sistema.

Em síntese, o *carpooling* representa uma solução concreta para os desafios da mobilidade urbana, ao integrar sustentabilidade ambiental, redução de impactos na saúde pública e fortalecimento comunitário. Com suporte tecnológico adequado e políticas públicas direcionadas, essa prática tem potencial para transformar positivamente a dinâmica urbana. A adoção de aplicativos por parte de instituições e comunidades reforça ainda mais sua viabilidade e aplicabilidade.

Esta revisão da literatura evidencia, portanto, que o *carpooling* não apenas responde às demandas contemporâneas por um transporte mais eficiente, como também favorece a construção de cidades mais sustentáveis, colaborativas e saudáveis.

2.2 ECONOMIA COLABORATIVA E APLICATIVOS DE MOBILIDADE

A economia colaborativa, também conhecida como economia do compartilhamento, tem causado uma transformação significativa em diversos setores, incluindo o transporte. Através da utilização de tecnologias digitais, plataformas como *Uber*, *BlaBlaCar* e *Lyft* têm demonstrado como o compartilhamento de recursos pode ser uma solução viável e eficiente para os problemas de mobilidade urbana. Essas plataformas conectam motoristas e passageiros de maneira inovadora, oferecendo uma alternativa conveniente e, muitas vezes, mais acessível aos sistemas de transporte tradicionais. Estudos mais recentes, como o de Hamari, Sjöklint e Ukkonen (2016), exploram como essas plataformas otimizam o uso de recursos e promovem uma forma de consumo mais sustentável.

A ascensão dos aplicativos de mobilidade é um fenômeno recente, mas com um impacto profundo e duradouro na forma como as pessoas se deslocam nas cidades. Tzimos (2021) discute como a economia colaborativa no setor de transporte não apenas promove a redução de custos para os usuários, mas também oferece

flexibilidade ao permitir que os usuários agendem corridas sob demanda, com pagamento integrado na plataforma, o que simplifica e melhora a experiência do usuário.

Um dos elementos fundamentais para o sucesso de aplicativos de mobilidade é a capacidade de estabelecer um vínculo de confiança entre os usuários. Embora estudos, como o de Ert, Fleischer e Magen (2016), destaquem que sistemas de avaliação e *feedback* têm papel importante na criação de ambientes mais transparentes e responsáveis, este aplicativo adotará outras estratégias para garantir a segurança e a credibilidade do serviço. Em vez de avaliações, a confiança será promovida por meio da implementação de protocolos rigorosos de segurança, incluindo a verificação de antecedentes dos motoristas e o uso de medidas de proteção voltadas ao passageiro. Tais práticas, amplamente utilizadas por plataformas consolidadas, contribuem significativamente para um ambiente mais seguro e confiável, mesmo na ausência de um sistema de notas ou comentários entre usuários.

Além dos benefícios em termos de conveniência e segurança, os aplicativos de mobilidade também têm um impacto econômico significativo. Eles criaram novas oportunidades de emprego e renda para milhares de motoristas ao redor do mundo. Cohen e Sundararajan (2015) destacam que, ao oferecer uma alternativa mais acessível ao transporte tradicional, esses aplicativos também contribuem para a inclusão social, permitindo que pessoas de diferentes faixas de renda tenham acesso a serviços de transporte de qualidade. Essa democratização do acesso ao transporte é um aspecto central da contribuição dos aplicativos de mobilidade para as cidades contemporâneas.

Outro ponto relevante é a integração dos aplicativos de mobilidade com outras formas de transporte, como ônibus e metrô, o que pode contribuir para a criação de um sistema de transporte público mais eficiente e coeso. (SHAHEEN et al., 2015) sugerem que essa integração pode ser facilitada por meio de parcerias estratégicas entre empresas de tecnologia e governos locais. Tais parcerias são fundamentais para o desenvolvimento de soluções de mobilidade que atendam às necessidades específicas das comunidades, tornando o sistema de transporte urbano mais eficaz e acessível para todos.

Assim, a economia colaborativa, através dos aplicativos de mobilidade, não apenas transforma a maneira como as pessoas se deslocam, mas também promove uma série de benefícios econômicos, sociais e ambientais, destacando-se como uma ferramenta poderosa para a modernização dos sistemas de transporte urbano.

2.3 TECNOLOGIAS DE DESENVOLVIMENTO PARA APLICATIVOS DE CARONA

A escolha das tecnologias adequadas é fundamental para o desenvolvimento e sucesso de um aplicativo de caronas, especialmente considerando as necessidades de segurança, escalabilidade e eficiência. Dentre as diversas opções disponíveis, o Firebase destaca-se como uma plataforma abrangente de desenvolvimento de aplicativos móveis e *web*, oferecendo uma vasta gama de serviços que facilitam o processo de criação e manutenção de aplicações robustas.

O Firebase é conhecido por fornecer uma infraestrutura escalável e segura, permitindo que os desenvolvedores se concentrem nas funcionalidades principais do aplicativo sem a preocupação com a complexidade da infraestrutura subjacente. Um dos principais componentes do Firebase, o Firebase Authentication, oferece suporte a múltiplos métodos de login, como email/senha, *Google*, *Facebook*, entre outros provedores de identidade. Essa flexibilidade é crucial para garantir uma experiência

de usuário fluida e segura, essencial para a confiança e adoção de um aplicativo de caronas. (GOOGLE, 2025a)

Outro serviço fundamental oferecido pelo Firebase é o Firebase Realtime Database, que permite o armazenamento e sincronização de dados em tempo real. Essa funcionalidade é particularmente importante para aplicativos de caronas, que exigem atualizações instantâneas para gerenciar as ofertas e solicitações de carona de maneira eficiente e eficaz. A capacidade de atualização em tempo real assegura que os usuários tenham sempre acesso às informações mais recentes, melhorando a usabilidade e a confiabilidade do aplicativo. (GOOGLE, 2025b)

Para ampliar o alcance e a usabilidade entre diferentes perfis de usuários de dispositivos móveis, é vantajoso desenvolver o aplicativo tanto para o sistema Android quanto para o iOS. Ferramentas *no-code*, como o MIT App Inventor, permitem a criação de aplicativos de forma intuitiva e acessível, facilitando a prototipagem e a implementação ágil de soluções móveis, sem a necessidade de conhecimentos avançados em programação.

No desenvolvimento em questão, o Firebase é utilizado para o gerenciamento do *backend* da aplicação. Já o App Inventor é empregado na criação da interface (*frontend*) e de parte das funcionalidades do *backend*, com integração ao Firebase (MIT APP INVENTOR COMMUNITY, 2025).

A integração dessas tecnologias — Firebase, MIT App Inventor — possibilita a criação de um aplicativo de caronas robusto, seguro e eficiente. A combinação dessas ferramentas oferece uma solução completa, cobrindo desde a autenticação de usuários até o armazenamento e a recuperação de dados em tempo real, e também o gerenciamento de informações localmente no dispositivo. Essa abordagem modular facilita a manutenção do aplicativo e permite a expansão de funcionalidades conforme necessário, sem comprometer a performance ou a segurança da aplicação. Embora o presente trabalho não tenha realizado testes com múltiplos usuários simultâneos, a infraestrutura do Firebase oferece suporte a esse tipo de crescimento, permitindo o atendimento a um número elevado de usuários em projetos futuros.

Assim, a escolha cuidadosa dessas tecnologias e sua integração harmoniosa no desenvolvimento do aplicativo de caronas resultam em uma solução tecnológica eficaz, que atende às demandas tanto dos usuários quanto dos desenvolvedores, promovendo uma experiência de uso fluida e segura, essencial para o sucesso de plataformas de economia colaborativa no setor de mobilidade.

3. METODOLOGIA

O processo de desenvolvimento do aplicativo segue uma abordagem incremental e iterativa, começando com o planejamento, em que são definidos os requisitos funcionais e não funcionais, bem como as tecnologias a serem utilizadas.

A camada de apresentação, desenvolvida no MIT App Inventor, será responsável pela interface do usuário, permitindo todas as interações com o aplicativo: cadastro, login, oferta e busca de caronas, além do gerenciamento de transportes. A camada de lógica de negócios também será renovada no MIT App Inventor em integração com o Firebase, utilizando blocos de programação para controlar a autenticação de usuários, validação de dados e as funções relacionadas às caronas.

No App Inventor, variáveis globais armazenam dados temporariamente usados pelo aplicativo, como o identificador único do usuário (idToken), URLs do Firebase para registro e login, entre outros.

Já a camada de dados, utilizar-se-á o Firebase. O Firebase Authentication será configurado para gerenciar o cadastro e autenticação dos usuários, enquanto o Firebase Realtime Database armazenará informações como ofertas de carona, perfis de usuários e dados de usuários.

A lógica de negócios do sistema cobre registro, login, cadastro de caronas e gerenciamento de ofertas. Durante o registro ou login, os dados serão enviados ao Firebase em requisições HTTP, cujas respostas em JSON permitem extrair informações como o idToken. No cadastro de caronas, as validações garantirão que as informações sejam completas e corretas antes de serem armazenadas no Firebase.

Diversos componentes suportarão o funcionamento do sistema. O Firebase Authentication garantirá segurança no login com vários métodos, enquanto o Realtime Database manterá os dados sincronizados entre dispositivos conectados. O TinyDB armazenará informações locais, como índices de caronas recentes, possibilitando acesso rápido sem internet. Mensagens contextuais serão gerenciadas pelo componente de notificação, aprimorando a experiência do usuário com feedback em tempo real.

Após o desenvolvimento, uma fase de testes garantirá a qualidade do aplicativo, com testes funcionais verificando as principais funcionalidades, como cadastro de usuários, oferta e busca de caronas, e testes de usabilidade para garantir uma interface intuitiva. A implantação envolve a preparação do ambiente de produção no Firebase, permitindo o lançamento do aplicativo. Por fim, a manutenção e as atualizações incluem o monitoramento do desempenho, a correção de possíveis *bugs* e a adição de novas funcionalidades baseadas no *feedback* dos usuários. Com esse processo estruturado, o desenvolvimento do aplicativo é reestruturado de forma organizada, garantindo que o produto final atenda às expectativas tanto dos desenvolvedores quanto dos usuários.

4. DESENVOLVIMENTO

Nesta seção, apresentamos o aplicativo tela a tela, detalhando sua interface, programação em blocos e interação com o Firebase. As telas principais e seus respectivos componentes são explicados em sequência, destacando sua relevância no fluxo do aplicativo. Cada tela é descrita com detalhes sobre a disposição dos elementos visuais e a funcionalidade que cada um oferece, garantindo uma compreensão clara do papel de cada componente no contexto geral do aplicativo. Iniciamos com a tela de login, que integra autenticação via Firebase, seguida pela tela principal que serve como *hub* central para acesso às funcionalidades principais. Detalhamos também as interações possíveis em cada tela, incluindo botões, campos de entrada de texto, e outros *widgets* interativos.

A primeira interação do usuário ocorre na tela de login, onde campos de entrada capturarão o e-mail e a senha. Botões específicos permitirão o registro de um novo usuário ou a autenticação de um já existente. Mensagens de erro ou sucesso serão exibidas por meio de um componente de notificação.

Após o login, o usuário será direcionado à tela de seleção das ações desejadas: cadastrar ou selecionar carona. Na tela de ofertas de caronas, visualizará, em tempo real, as caronas disponíveis, carregadas diretamente do Firebase. Um menu suspenso permitirá a seleção de uma carona, e *pop-ups* exibem os detalhes para confirmação da escolha.

Caso o usuário também deseje oferecer caronas, ele poderá acessar a tela de cadastro, destinada à criação de novas ofertas.

Na tela de cadastro de caronas, o usuário poderá registrar novas ofertas, preenchendo campos como horário, número de vagas e sentido da viagem. Os dados serão validados antes de serem enviados ao Firebase, sendo armazenados sob uma *tag* específica. Após o cadastro, mensagens de confirmação serão exibidas ao usuário, garantindo transparência no processo.

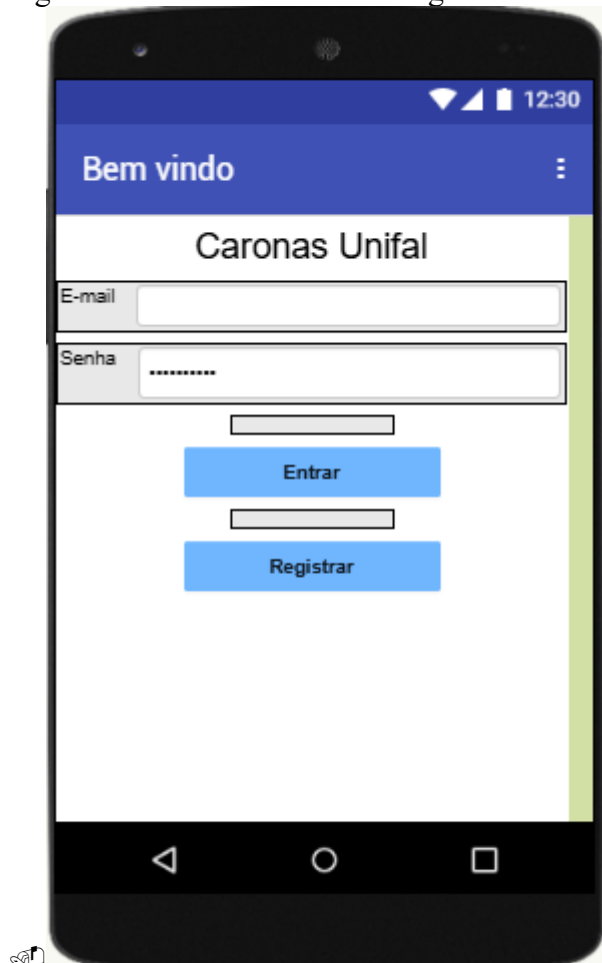
A programação em blocos é explicada, mostrando como os blocos lógicos são organizados para manipular dados de entrada do usuário e interagir com o banco de dados Firebase. Esse detalhamento inclui exemplos de código e explicações sobre como os dados são recuperados, manipulados e armazenados. A integração com o Firebase é destacada, demonstrando como o aplicativo utiliza este serviço para armazenamento de dados em nuvem, autenticação de usuários e outras operações *backend*. Esta seção fornece *insights* sobre configurações de segurança, gestão de sessão de usuários e sincronização de dados em tempo real.

Com essa abordagem, pretendemos fornecer uma visão completa da construção do aplicativo, desde a interface gráfica até a lógica de programação e integração com serviços de *backend*, proporcionando uma base sólida para o entendimento e replicação do projeto.

4.1 TELA DE LOGIN E REGISTRO (SCREEN 1)

A primeira tela, exibida na Figura 1, é a tela de entrada do aplicativo, projetada para facilitar o acesso dos usuários, nela contém campos de entrada para **E-mail** e **Senha**, ambos conectados ao sistema de autenticação do Firebase. Os botões "Registrar" e "Entrar" permitem que o usuário escolha entre criar uma nova conta ou acessar uma existente..

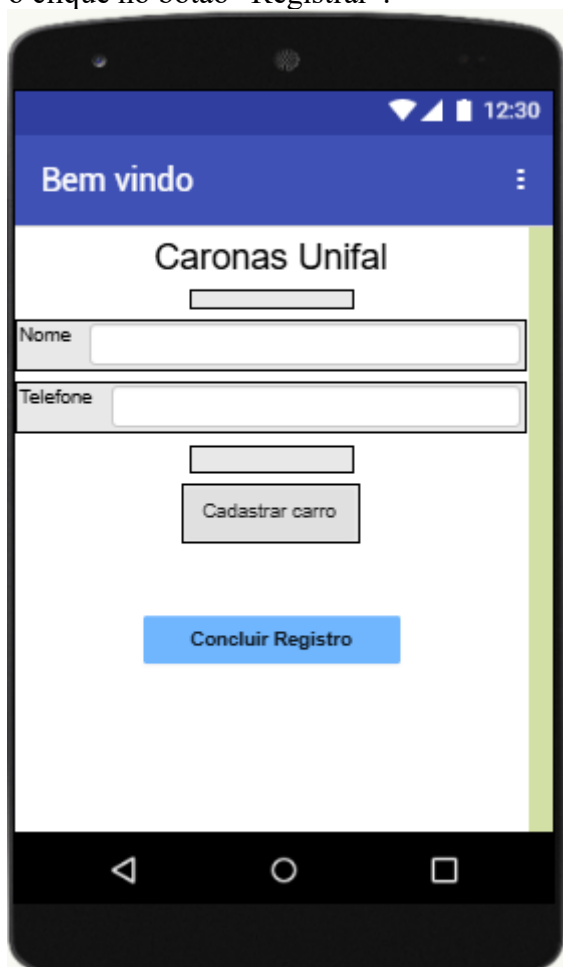
Figura 1 - Tela Inicial : tela de login e cadastro de novos usuários.



Ao pressionar o botão “Registrar”, o sistema inicia automaticamente o processo de criação de uma nova conta no Firebase, gerando um identificador único (User ID) mesmo antes do preenchimento completo dos dados. Diferentemente de um redirecionamento para outra tela, o aplicativo mantém o usuário na mesma interface, tornando visíveis novos campos anteriormente ocultos. Esses campos incluem o nome completo, número de telefone e a opção de cadastrar um veículo.

A Figura 2 ilustra essa etapa do processo, com os campos adicionais de cadastro sendo apresentados ao usuário após o clique no botão de registro. Essa abordagem visa proporcionar uma navegação mais fluida e centralizada, mantendo todas as ações de registro unificadas em uma única tela.

Figura 2 - Tela com campos de nome, telefone e botão para cadastro de veículo, exibidos após o clique no botão “Registrar”.

The image shows a smartphone screen with a mobile application interface. At the top, there is a blue header bar with the text "Bem vindo" on the left and a three-dot menu icon on the right. Below the header, the text "Caronas Unifal" is centered. Underneath, there is a small, empty rectangular input field. This is followed by two labeled input fields: "Nome" and "Telefone". Below these, there is another small, empty rectangular input field. Then, there is a button labeled "Cadastrar carro". At the bottom of the form area, there is a blue button labeled "Concluir Registro". The smartphone's status bar at the very top shows signal strength, battery level, and the time "12:30". The bottom of the screen shows the standard Android navigation bar with back, home, and recent apps icons.

Além do preenchimento dos dados pessoais, o usuário pode optar por cadastrar seu veículo. Ao clicar no botão “Cadastrar carro”, novos campos se tornam visíveis na mesma interface, permitindo o preenchimento das informações do automóvel, como modelo, cor e placa. A interface também inclui a possibilidade de registrar os pontos de parada usuais no trajeto até a UNIFAL, otimizando o planejamento das caronas oferecidas.

A Figura 3 apresenta essa etapa do processo, na qual os campos adicionais se expandem dinamicamente.

Figura 3 - Tela com campos de cadastro do veículo e definição de pontos de parada, exibidos após o clique em “Cadastrar carro”.

The image shows a smartphone screen with a blue header bar containing the text "Bem vindo" and a hamburger menu icon. Below the header is a small white input field. The main content area contains a form with three input fields labeled "Carro", "Cor", and "Placa". Below these is a section titled "Pontos de parada usuais, sentido UNIFAL" with a label "Ponto de partida quando vai a Unifal:" and an input field. A grey button labeled "Adicionar" is positioned below the input field. At the bottom of the form is a blue button labeled "Concluir Registro". The screen has a black Android navigation bar at the bottom.

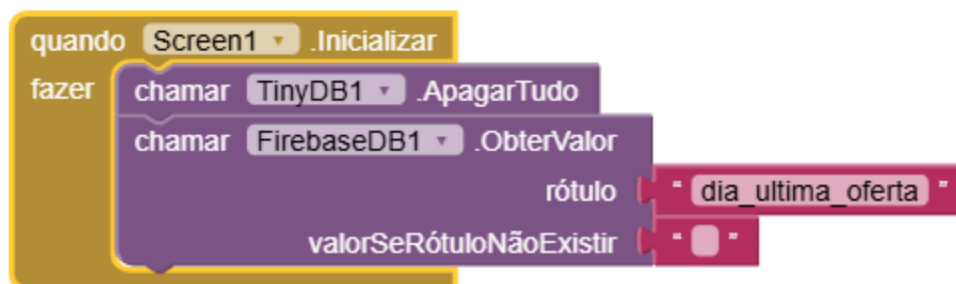
Essa abordagem reduz a complexidade da navegação e melhora a experiência do usuário, concentrando as etapas iniciais do cadastro em uma única sequência interativa.

4.1.1 Implementação das Funcionalidades da Screen1

Blocos de Inicialização de tela

A tela Screen1 inicializa limpando o TinyDB e recuperando do Firebase a data da última oferta de carona, vide Figura 4.

Figura 4 - Bloco de inicialização da Screen 1.



Blocos de Inicialização e Login

Inicialização de Variáveis Globais

Na Figura 5, são apresentados os blocos de inicialização de variáveis para a Screen 1.

- inicializar global id: Essa variável global será utilizada para armazenar um identificador único do usuário.
- inicializar global URLRegistrar e inicializar global URLIngresar: Essas variáveis armazenam URLs que serão usadas para realizar o cadastro e o login de usuários. Elas estão associadas ao serviço de autenticação do Firebase, como indicado pelas URLs que apontam para endpoints do Firebase.

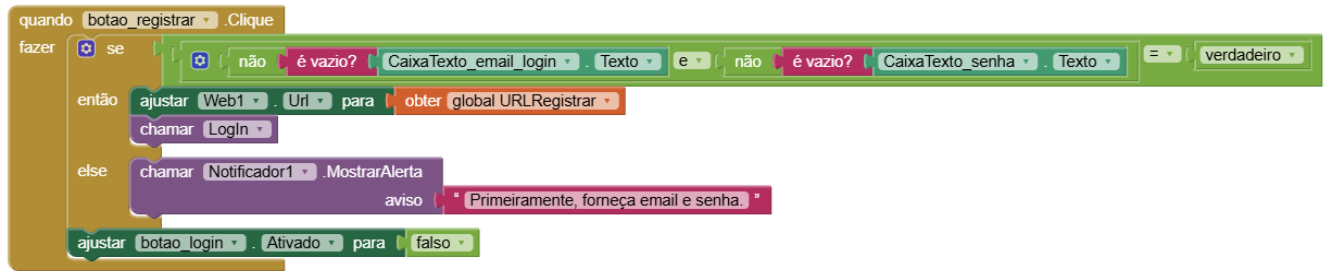
Figura 5 - Blocos que definem variáveis globais e URLs do Firebase para registro e login de usuários.



Botão de Registro (botao_registrar . Clique)

Este bloco, vide Figura 6, é executado quando o botão de registro é clicado. Este bloco faz uma chamada para o bloco Login, passando a URL de registro, iniciando o processo de registro de um novo usuário com base em futuras caixas que serão ativadas, como veremos na Figura 6, onde será registrada as credenciais inseridas.

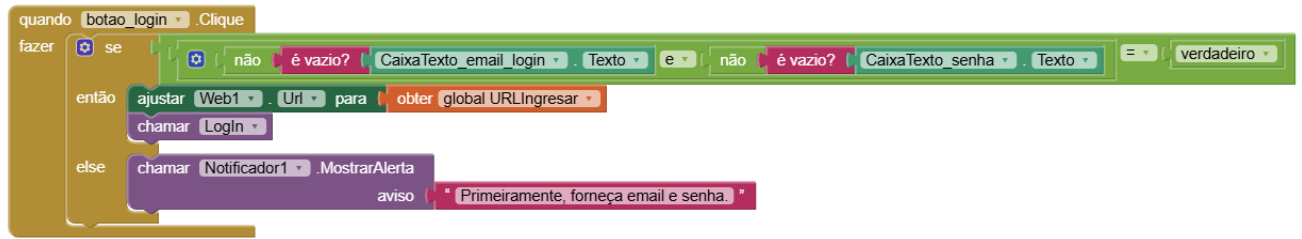
Figura 6 - Códigos para processo de registro de novo usuário.



Botão de Login (botao_login . Clique)

Similar ao botão de registro, este bloco faz uma chamada para o bloco Login, mas passa a URL de login, iniciando o processo de autenticação do usuário, vide Figura 7.

Figura 7 - Códigos para processo de autenticação do login.



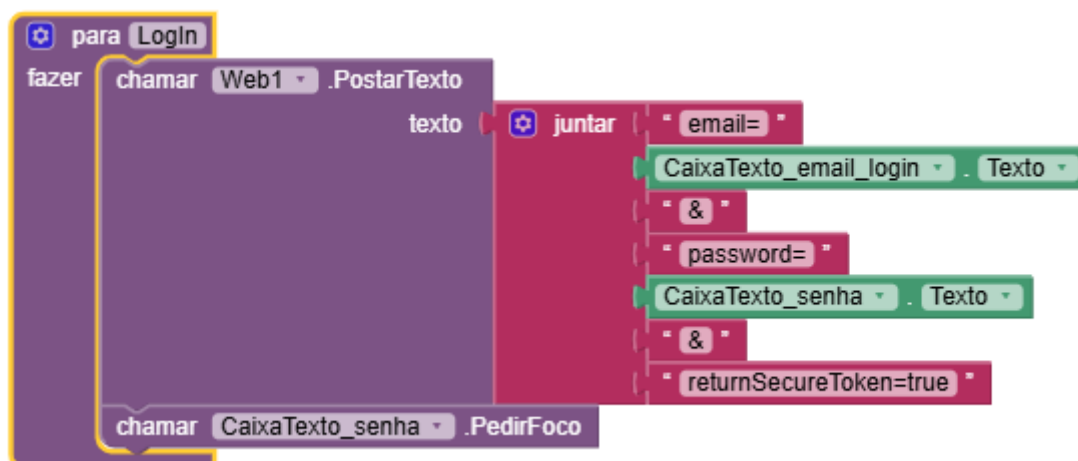
Bloco de Login (para Login)

Este bloco, vide Figura 8, executa a ação de login ou registro dependendo da URL fornecida. A lógica para registro/login inclui a captura de dados inseridos pelo usuário e o envio dessas informações ao Firebase Authenticator.

Ao “Postar Texto para Web1”:

- O texto a ser enviado para o serviço do Firebase é uma combinação dos campos de entrada para email e *password*.
- O bloco utiliza a função *join* para concatenar o email e senha no formato necessário para a requisição HTTP.
- Após preparar o texto, ele chama a função PedirFoco para colocar o foco no campo apropriado.

Figura 8 - Códigos para executar a ação de login ou registro.

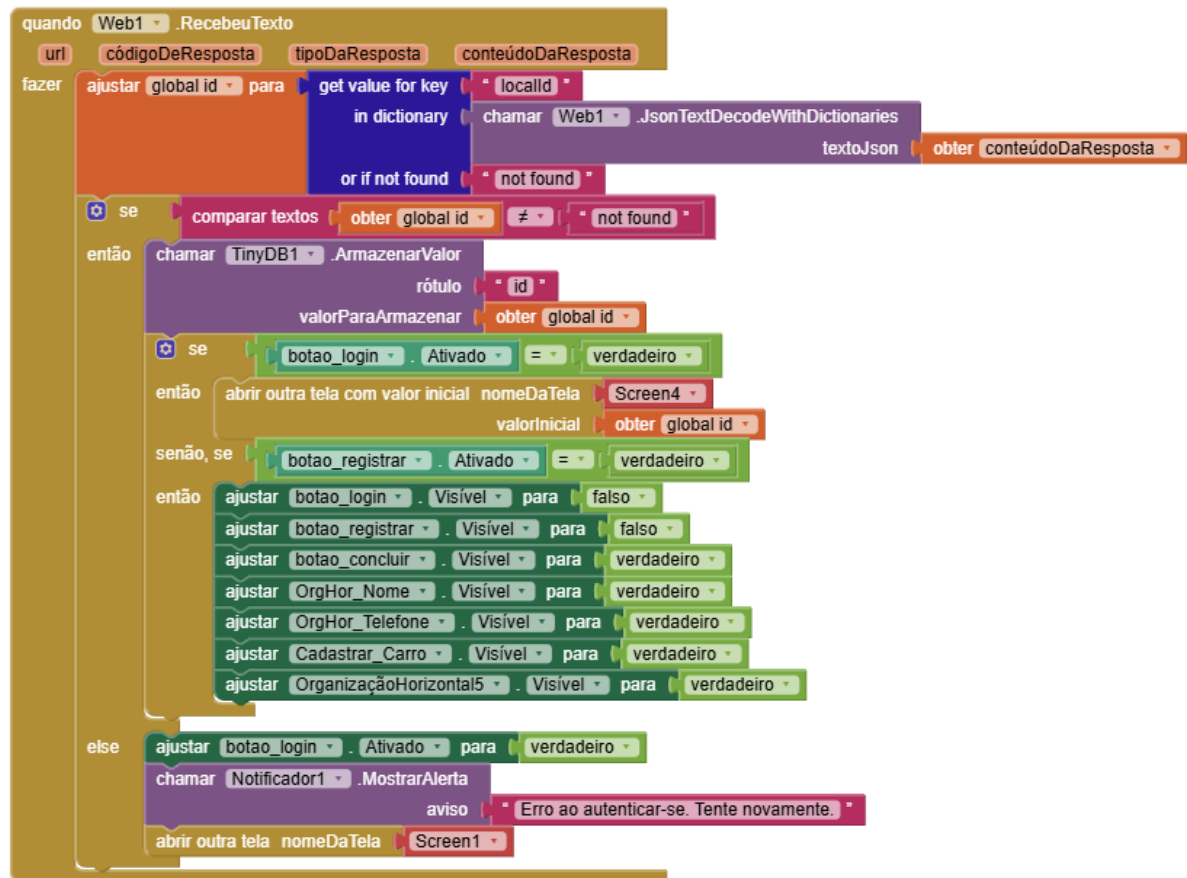


Bloco de Resposta ao Login (quando Web1 . RecebeuTexto)

Este bloco, vide Figura 9, lida com a resposta que o aplicativo recebe após enviar a requisição de login:

- **Decodificação da Resposta JSON:**
 - O texto recebido é tratado como um JSON e decodificado para extrair informações.
 - Se um *idToken* for encontrado, isso indica que a autenticação foi bem-sucedida.
- **Armazenamento do *idToken* e Login Bem-Sucedido:**
 - O *idToken*, representado pela chave *localid*, é armazenado na variável global *id*.
 - Se for um processo de login, o aplicativo direciona o usuário para a *Screen4*.
 - Senão, o bloco chama a função *gravar_usuario* para armazenar os detalhes do usuário no banco de dados *Firebase*.
- **Em caso de Falha:**
 - Exibe-se uma mensagem de erro e a *Screen1* é recarregada.

Figura 9 - Blocos que validam o login via Firebase e ajustam a interface conforme o perfil do usuário.



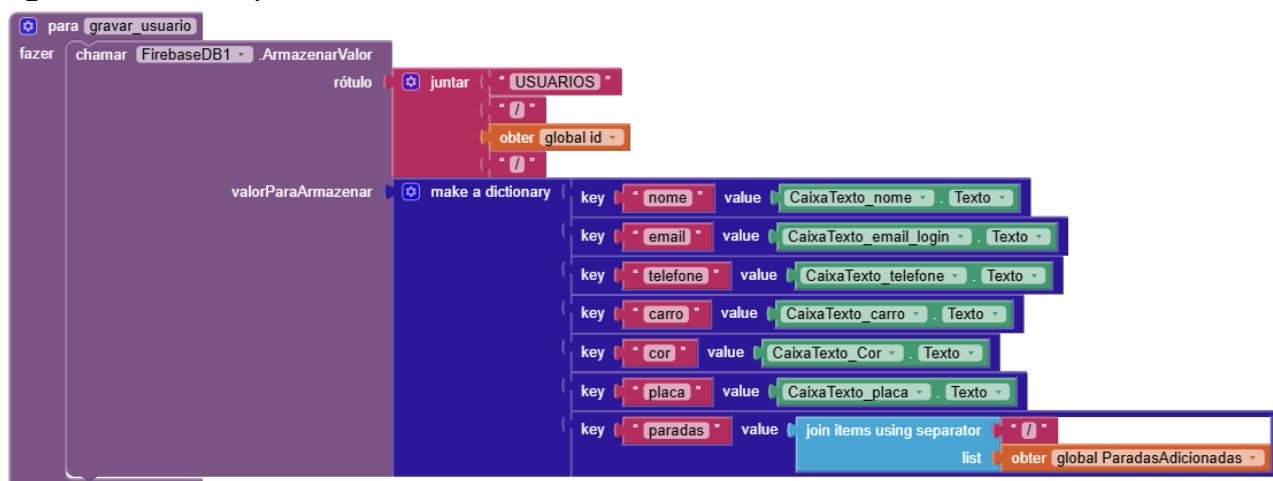
Bloco de Armazenamento de Dados do Usuário (para gravar_usuario)

Este bloco, representado na Figura 10, lida com o armazenamento dos dados do usuário no banco de dados Firebase.

- **Armazenamento de Valores no Firebase:**

Os dados do usuário, como email, telefone, nome, carro, cor, placa, senha, paradas, e bairro, são armazenados como um dicionário no Firebase Database, sob a *tag* USUARIOS, e identificado pelo id do usuário. Este dicionário contém as informações do usuário necessárias para o funcionamento do aplicativo, como as informações do veículo para motoristas ou as paradas preferidas para passageiros.

Figura 10 - Blocos que salvam os dados do usuário no Firebase em formato de dicionário.

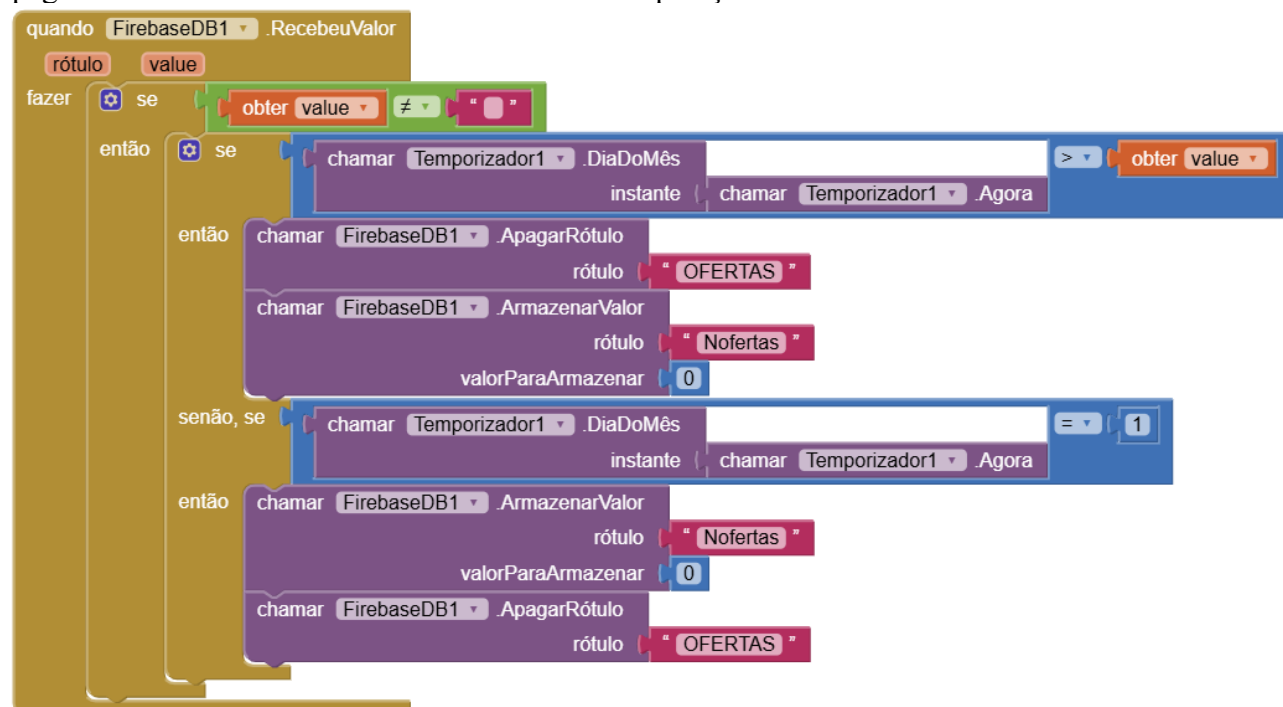


Bloco de Recebimento de Valores do Firebase (quando Firebase . RecebeuValor)

Este bloco, Figura 11, trabalha para que o dicionário que é contido as ofertas seja limpo a cada dia útil.

O bloco apresentado é responsável por verificar, ao receber um valor do Firebase, se esse valor corresponde ao dia atual do mês. Se não for o mesmo dia, o sistema apaga o rótulo "OFERTAS" e limpa o campo "Nofertas".

Figura 11- Blocos que controlam a atualização de ofertas no Firebase com base na data atual, apagando ou armazenando valores conforme a comparação.

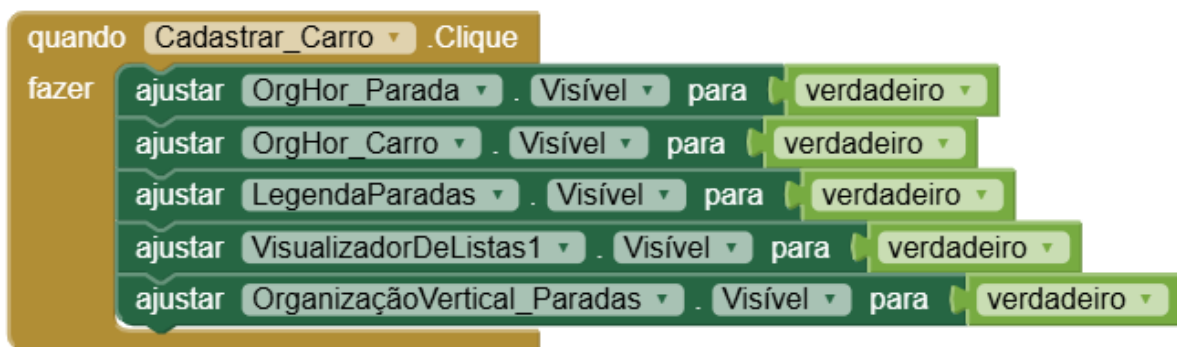


Bloco de Cadastro de carros

Como podemos observar o bloco pela Figura 12, quando o botão Cadastrar_Carro é clicado, aciona-se o procedimento de LogIn que consequentemente aciona os campos para inserção dos dados do carro que será usado para ofertar a carona, de forma que:

- Configura visibilidade e valores padrão para campos de entrada como cor do veículo, placa, modelo.
- Torna a tela mais dinâmica, exibindo os campos necessários apenas quando um botão específico é acionado.
- Permite a personalização do cadastro de acordo com as informações previamente salvas no Firebase.

Figura 12 - Blocos que tornam visíveis os componentes da interface ao clicar no botão "Cadastrar Carro".

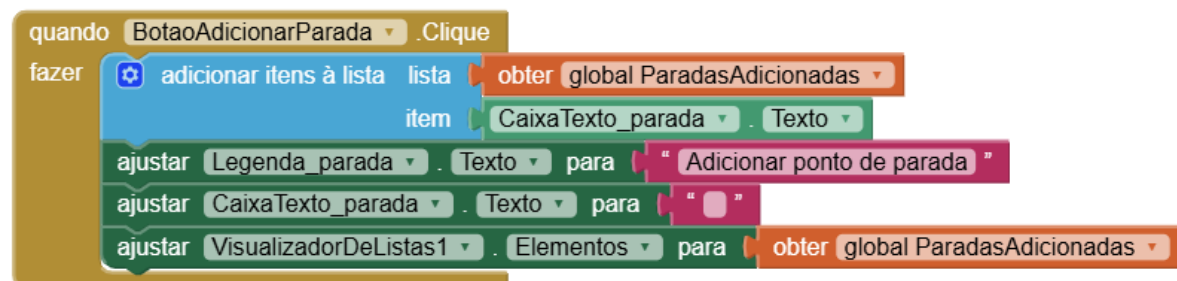


Bloco de inclusão de paradas

O bloco BotãoAdicionarParada mostrado na Figura 13, quando clicado implementa a lógica para gerenciar paradas intermediárias:

- Permite adicionar múltiplas paradas a uma lista.
- Usa variáveis globais para armazenar informações como nome e detalhes da parada.
- Atualiza dinamicamente a interface com as novas paradas.

Figura 13 - Blocos que adicionam um novo ponto de parada à lista e atualizam a interface ao clicar no botão "Adicionar Parada".

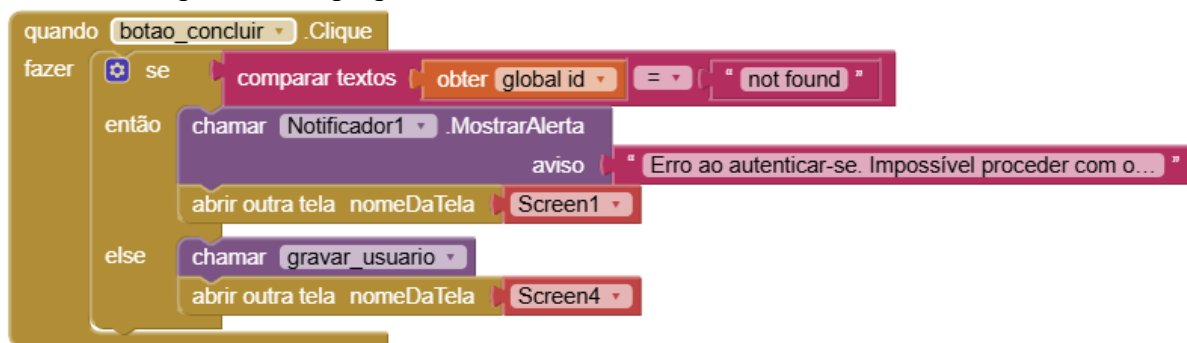


Bloco do botão concluir

O botão "Concluir" no fluxo do aplicativo desempenha a função de finalizar a etapa de cadastramento de novo usuário, garantindo que todas as ações relacionadas ao processo anterior sejam concluídas de forma adequada, vide Figura 14.

Se o usuário foi corretamente registrado no Firebase, então, chama-se o bloco de gravação dos dados do usuário no Firebase visto na Figura 8, para que com os detalhes de paradas e carro seja juntamente registrados. Ao fim da gravação, o usuário é direcionado para a Screen4. Caso contrário, o usuário é notificado do erro e a Screen1 é recarregada.

Figura 14 - Blocos que verificam a autenticação do usuário ao clicar em "Concluir" e redirecionam para a tela apropriada ou exibem um alerta em caso de erro.



4.2 INTERFACE DE USUÁRIO(SCREEN 4)

A Screen 4, como podemos observar na Figura 15, é uma das principais telas do aplicativo, responsável por gerenciar a navegação entre as funcionalidades centrais: oferta de caronas, solicitação de caronas e encerramento da aplicação. Essa tela foi projetada para garantir simplicidade e acesso rápido às principais ações do aplicativo. A interface de usuário mostrada na imagem contém três botões principais: Oferecer, Solicitar e Encerrar, cuja programação será detalhada a seguir.

Figura 15 - Tela do aplicativo com o título "Funcionalidades" e três botões: "Oferecer", "Solicitar" e "Encerrar", representando as principais ações disponíveis ao usuário.

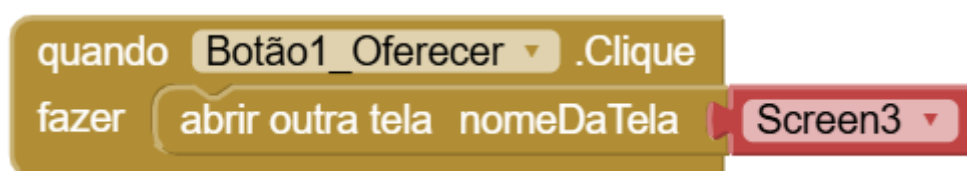


4.2.1 Implementação das Funcionalidades as Screen4

Botão "Oferecer"

Este botão apresentado na Figura 16, permite ao usuário oferecer uma carona. Quando clicado, ele direciona o usuário para a tela de oferta de caronas.

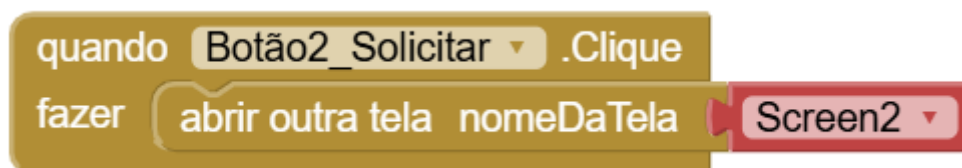
Figura 16 - Bloco que define a ação do botão "Oferecer", redirecionando o usuário para a tela Screen3 ao ser clicado.



Botão "Solicitar"

Este botão, mostrado na Figura 17, permite ao usuário solicitar uma carona. Ao ser clicado, o usuário é direcionado para a tela onde pode buscar uma carona disponível.

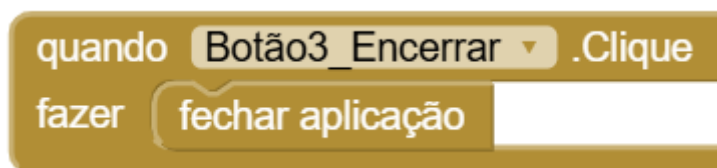
Figura 17 - Bloco que define a ação do botão "Solicitar", redirecionando o usuário para a tela Screen2 ao ser clicado.



Botão "Encerrar"

Veja na Figura 18 como esse botão apenas encerra a aplicação,.

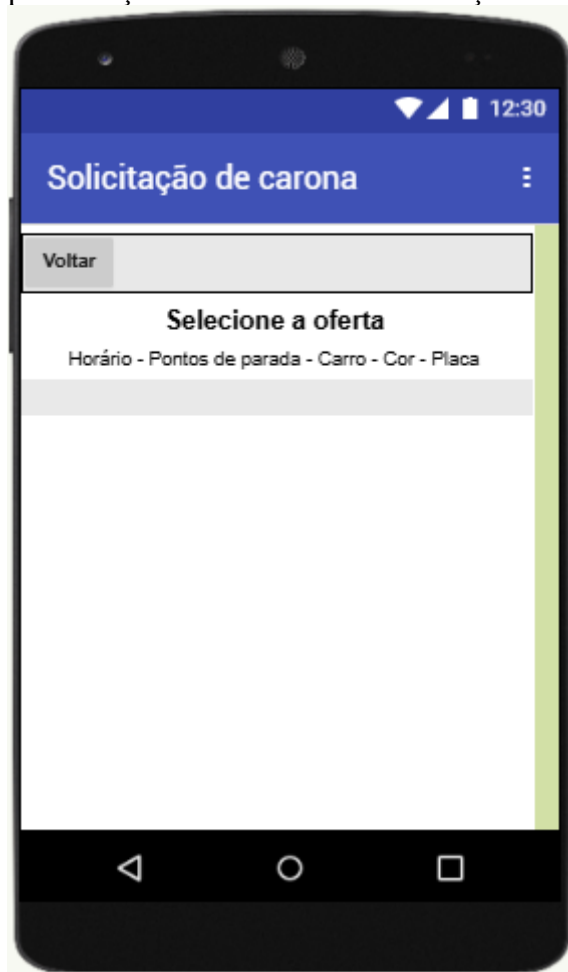
Figura 18- Bloco que define a ação do botão "Encerrar", finalizando a aplicação ao ser clicado.



4.3 INTERFACE DE SOLICITAÇÃO DE CARONA (SCREEN 2)

Na Screen 2, vide Figura 19, o layout é focado na solicitação de uma carona disponível.

Figura 19 - Tela "Solicitação de carona" do aplicativo, exibindo um botão "Voltar" e uma lista para seleção de ofertas com informações sobre horário, pontos de parada, carro, cor e placa.



4.3.1 Implementação das Funcionalidades as Screen2

Botão "Voltar"

Este botão permite ao usuário retornar à tela anterior. É útil para navegação dentro do aplicativo.

Menu Suspenso "Selecione a oferta"

Este menu suspenso (*Spinner*) permite que o usuário selecione uma carona disponível a partir de uma lista de ofertas. As opções disponíveis neste menu são carregadas a partir do banco de dados ou de uma fonte de dados interna.

Inicialização de Variáveis Globais

As variáveis globais `num_vagas`, `ofertas`, `cont`, `num_ofertas`, e `ofertas_detalhes` são inicializadas para armazenar informações sobre as caronas disponíveis, o número de ofertas, e os detalhes de cada oferta como podemos visualizar na Figura 20.

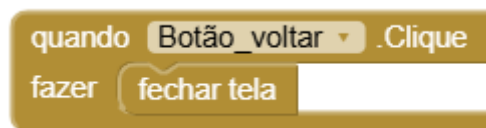
Figura 20 - Blocos de inicialização de variáveis globais utilizadas para controle e armazenamento de ofertas e vagas no aplicativo.



Bloco "Voltar" (Botão_voltar . Clique)

Quando o botão "Voltar" é clicado, ele chama o bloco para fechar a tela atual, retornando à tela anterior, Figura 21.

Figura 21 - Bloco que define a ação do botão "Voltar", fechando a tela atual ao ser clicado.



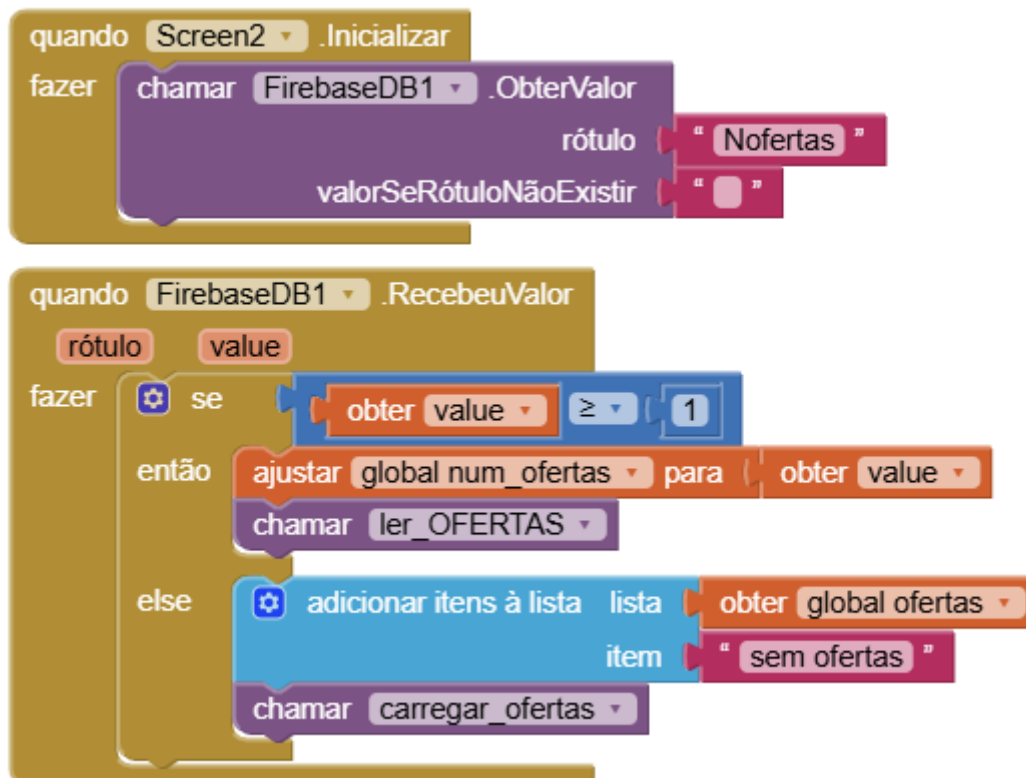
Recebimento de Valor do FirebaseDB1 (FirebaseDB1. ObterValor)

● Interação com o FirebaseDB no carregamento de ofertas:

- Na parte superior da Figura 22, o aplicativo realiza a solicitação de dados ao Firebase ao ser iniciado, buscando o valor associado ao rótulo "Nofertas". Caso esse rótulo não exista, será atribuído um valor vazio.
- Já na parte inferior da figura, é tratado o evento que ocorre ao receber os dados. Se o rótulo retornado for "num_ofertas" e houver ofertas disponíveis (valor maior que zero), o sistema chama a função ler_OFERTAS para carregar as informações completas. Caso contrário, exibe a mensagem "sem ofertas" na lista e chama a função carregar_ofertas para atualizar a interface exibida ao usuário.

Esse processo garante que o aplicativo busque, interprete e apresente dinamicamente as ofertas disponíveis por meio de um menu suspenso (Spinner), de forma simples e automática.

Figura 22 - Blocos responsáveis por obter e processar as ofertas ao iniciar a tela Screen2, verificando dados no Firebase e chamando funções conforme a disponibilidade de ofertas.

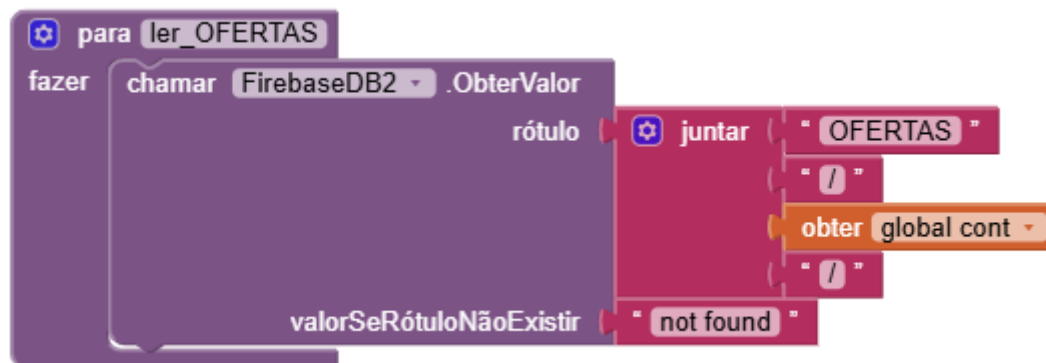


Recebimento de Valor do FirebaseDB (FirebaseDB2. Ler_ofertas)

O bloco apresentado na Figura 23 refere-se à função ler_OFERTAS, responsável por buscar informações detalhadas de cada carona registrada no banco de dados. A chamada é feita ao Firebase utilizando o rótulo composto por "OFERTAS" e um índice numérico armazenado na variável global cont. Caso o rótulo não exista, o valor padrão "not found" é retornado.

Essa etapa é fundamental para acessar dados como o número de vagas, nome do motorista e demais detalhes da carona. Após o recebimento da resposta, o evento FirebaseDB.RecebeuValor entra em ação para processar e armazenar essas informações em variáveis globais, permitindo sua reutilização em outras partes do aplicativo. Os dados também são organizados em uma lista que alimenta o componente Spinner, facilitando a seleção por parte do usuário.

Figura 23 - Bloco da função "ler_OFERTAS" que acessa o Firebase para obter os dados de uma oferta específica com base no contador global.



Recebimento de Valor do FirebaseDB (FirebaseDB2. RecebeuValor)

O bloco ilustrado na Figura 24 representa a lógica executada quando o aplicativo recebe os dados de uma oferta de carona a partir do Firebase Database, especificamente por meio do evento FirebaseDB2.RecebeuValor. Essa estrutura é responsável por processar cada oferta individualmente, extraindo seus dados e armazenando-os em variáveis globais para posterior exibição ao usuário.

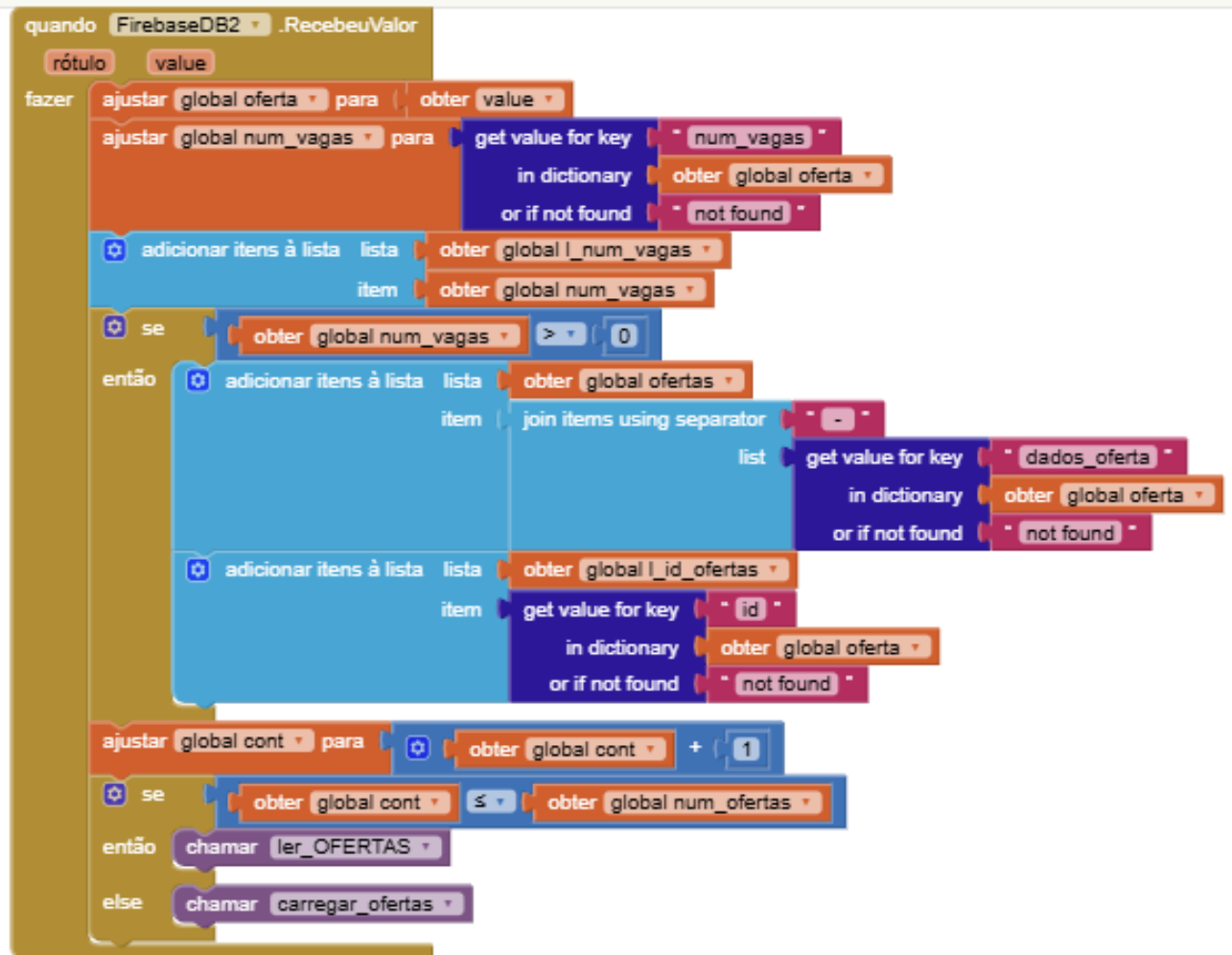
Inicialmente, o valor recebido do banco, que corresponde a uma oferta específica, é armazenado na variável global oferta. Em seguida, o sistema utiliza a chave "num_vagas" para acessar a quantidade de vagas disponíveis nessa oferta, armazenando o valor obtido na variável global num_vagas. Esse dado também é adicionado a uma lista geral que contém a quantidade de vagas de todas as ofertas, o que facilita o controle e a exibição conjunta dessas informações.

Na sequência, o bloco verifica se o número de vagas é diferente de zero. Caso a oferta ainda esteja disponível, o sistema adiciona à lista "global ofertas" os principais dados da carona, como ponto de partida, destino, horário e demais informações contidas na chave "dados_oferta". Além disso, o identificador único da carona (obtido pela chave "id") é adicionado à lista "global id_ofertas". Esses dados são posteriormente utilizados para alimentar o componente visual Spinner, permitindo a seleção da oferta pelo usuário.

Após o processamento da oferta atual, o aplicativo incrementa o valor da variável "global cont", que atua como um contador de controle, indicando quantas ofertas já foram processadas. Em seguida, o sistema verifica se o valor de cont ainda é menor que o número total de ofertas (global num_ofertas). Se houver mais ofertas a serem processadas, a função ler_OFERTAS é chamada novamente para buscar a próxima entrada. Caso todas as ofertas já tenham sido tratadas, a função carregar_ofertas é acionada para atualizar a interface do usuário com os dados finais.

Esse bloco é essencial para garantir que o aplicativo leia e organize corretamente os dados de caronas disponíveis, processando cada oferta de forma iterativa e eficiente, assegurando que apenas as caronas com vagas disponíveis sejam exibidas ao usuário de forma clara e organizada.

Figura 24 - Blocos que processam dados do FirebaseDB2 para carregar ofertas, vagas e IDs, atualizando listas e controlando a exibição com base no contador.



Procedimento de Carregar Oferta (carregar_ofertas)

A função `carregar_ofertas`, vide Figura 25, define os elementos do componente `VisualizadorDeListas1` com os dados armazenados na variável “global ofertas”. Esses dados representam as caronas disponíveis e são exibidos em formato de lista, permitindo que o usuário visualize e selecione uma das opções.

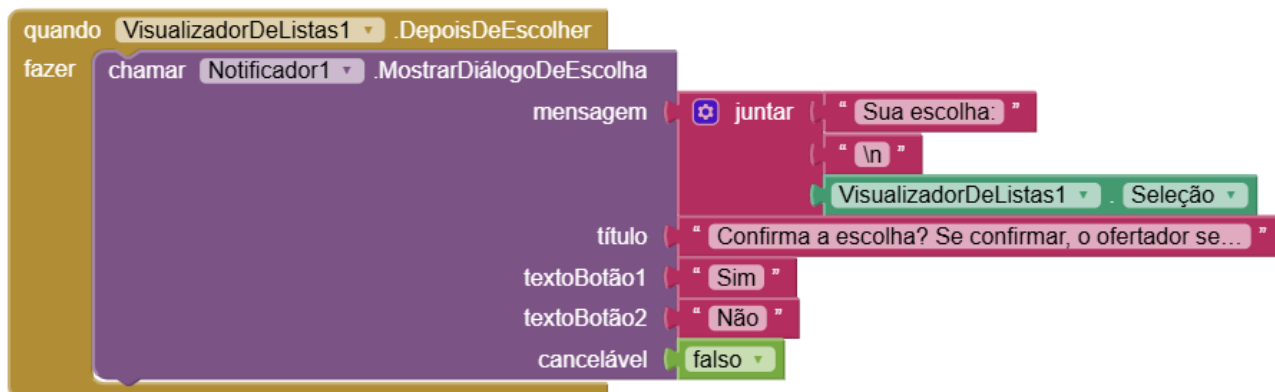
Figura 25 - Bloco da função “`carregar_ofertas`” que exibe as ofertas no componente de lista usando os dados armazenados na variável global.



Bloco de Seleção de Oferta (VisualizadorDeLista1 . DepoisDeEscolher)

Na Figura 26, quando o usuário seleciona uma oferta do menu suspenso, este bloco captura a seleção, processa os dados e exibe as informações detalhadas da carona escolhida.

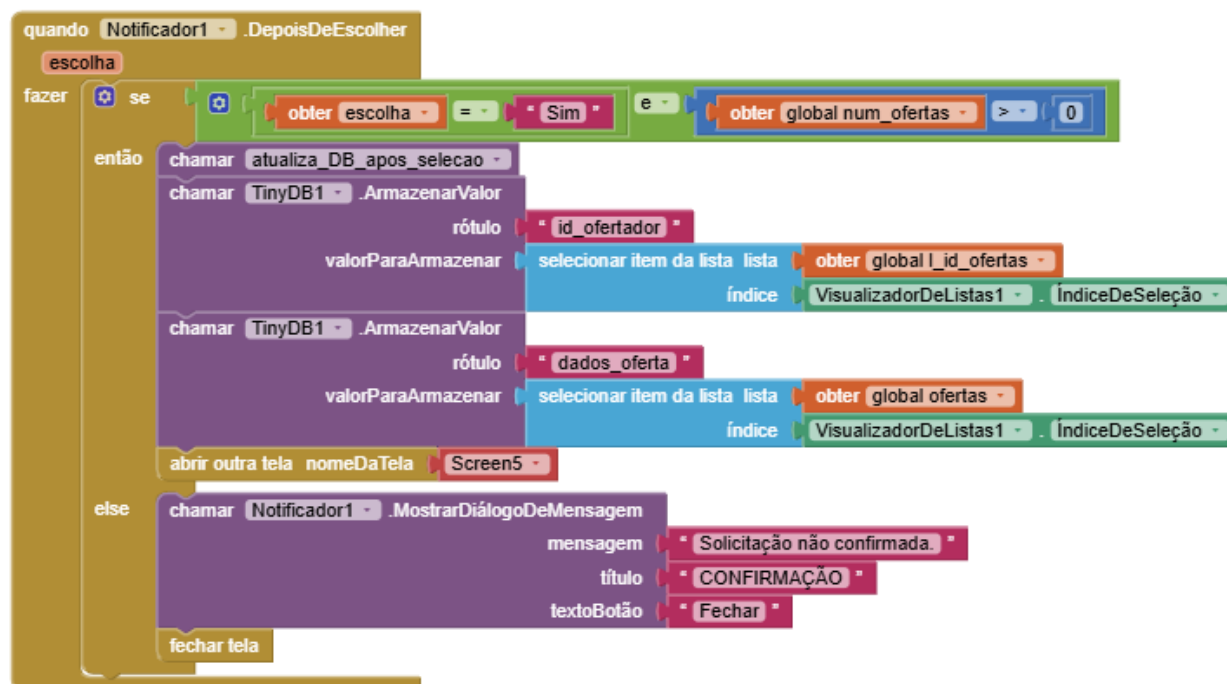
Figura 26 - Blocos que exibem um diálogo de confirmação ao usuário após a seleção de uma oferta na lista, mostrando a escolha e opções para confirmar ou cancelar.



Bloco de Confirmação de Seleção (Notificador1. DepoisDeEscolher)

O bloco da Figura 27 trata da confirmação de uma carona selecionada pelo usuário. Ao escolher "Sim" em uma janela de confirmação, o sistema verifica se há ofertas disponíveis. Caso positivo, executa uma função para atualizar o banco de dados e armazena localmente, via TinyDB, o identificador e os dados da carona escolhida, com base no item selecionado na lista. Em seguida, o aplicativo abre a tela "Screen5" para dar continuidade ao processo. Se o usuário não confirmar a seleção ou não houver ofertas, uma mensagem de aviso é exibida, informando que a solicitação não foi concluída.

Figura 27 - Blocos que confirmam a escolha de uma oferta, salvam os dados no TinyDB e redirecionam ou exibem mensagem conforme a resposta do usuário.



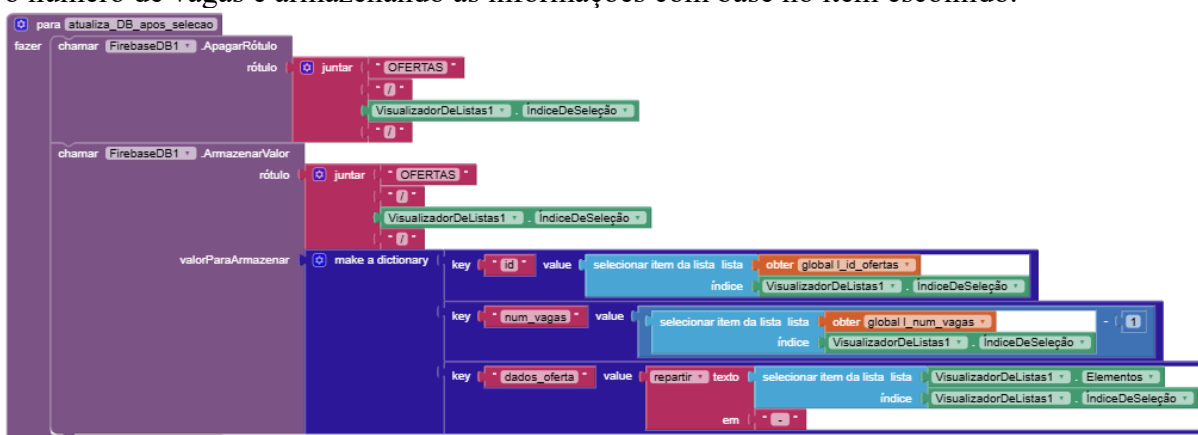
A função `atualiza_DB_apos_selecao`, representada na Figura 28, é executada logo após o usuário confirmar a escolha de uma carona. Essa função tem como

objetivo atualizar as informações no banco de dados Firebase, refletindo a nova quantidade de vagas disponíveis.

O processo começa com a remoção do rótulo referente à carona selecionada, utilizando a junção do texto "OFERTAS" com o índice do item selecionado na lista (VisualizadorDeListas1.IndiceDeSelecao). Em seguida, o mesmo rótulo é recriado com os dados atualizados, por meio do bloco ArmazenarValor.

A nova informação armazenada é um dicionário contendo três chaves: id, num_vagas, dados_oferta (local, paradas e horário), extraídos diretamente da lista de elementos exibida no componente de visualização (VisualizadorDeListas1.Elementos).

Figura 28 - Blocos que atualizam os dados de uma oferta no Firebase após a seleção, ajustando o número de vagas e armazenando as informações com base no item escolhido.



Abrir outra tela

O usuário confirma a solicitação da carona na Tela. A função atualiza_DB_apos_selecao é chamada para registrar os dados no Firebase e no TinyDB. O sistema redireciona automaticamente o usuário para a Tela 5 (Screen5), mostrado na Figura 29, onde ele poderá selecionar o ponto de parada para o embarque.

4.4 INTERFACE DE SELEÇÃO DE PONTO DE EMBARQUE E AVISO AO MOTORISTA (SCREEN5)

A tela de escolha do ponto de embarque, representada na Figura 29, oferece ao usuário uma interface intuitiva para selecionar o local onde deseja ser buscado. Essa funcionalidade é essencial para garantir que o trajeto da carona seja compatível com os pontos de parada previamente estabelecidos pelos motoristas. A interface utiliza um menu suspenso com os pontos disponíveis, permitindo uma seleção rápida e eficaz.

Figura 29 - Tela de seleção do ponto de parada para embarque.



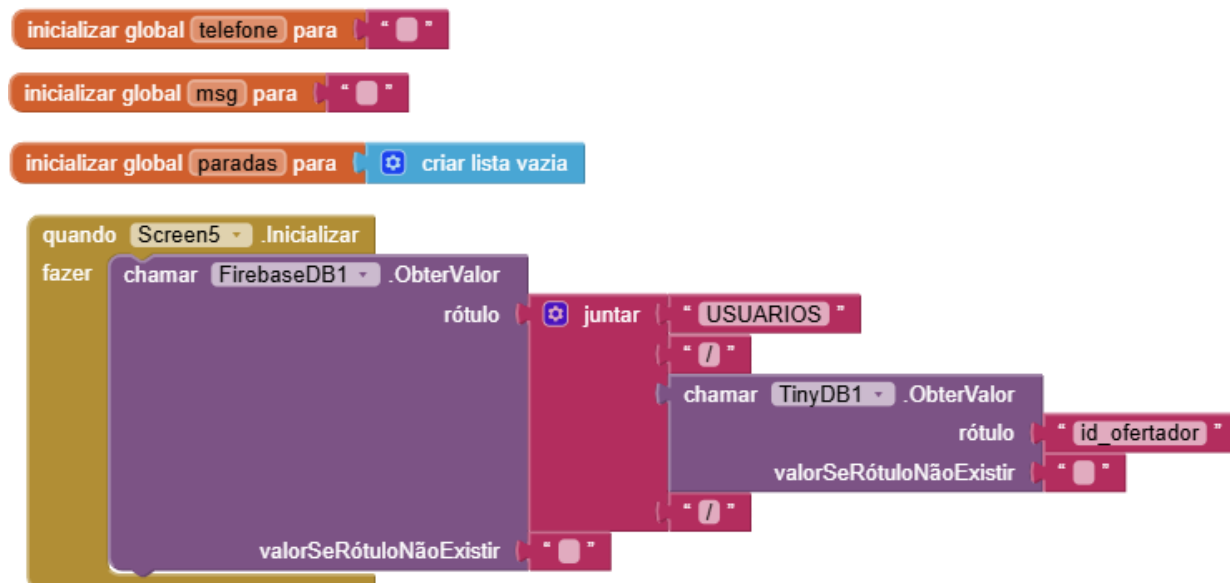
4.4.1 Implementação das Funcionalidades na Screen5

Inicialização da Tela

Ao abrir a Screen 5, vide Figura 30, ocorre a busca dos dados do motorista e das paradas armazenadas no Firebase.

- O FirebaseDB busca os dados do motorista a partir do identificador armazenado no TinyDB quando a carona foi selecionada.
- As variáveis globais telefone, msg e paradas são inicializadas para armazenar os dados do motorista e os pontos de parada disponíveis.

Figura 30 - Blocos que inicializam variáveis globais e, ao abrir a Screen5, obtêm dados do Firebase com base no identificador do ofertador armazenado no TinyDB.

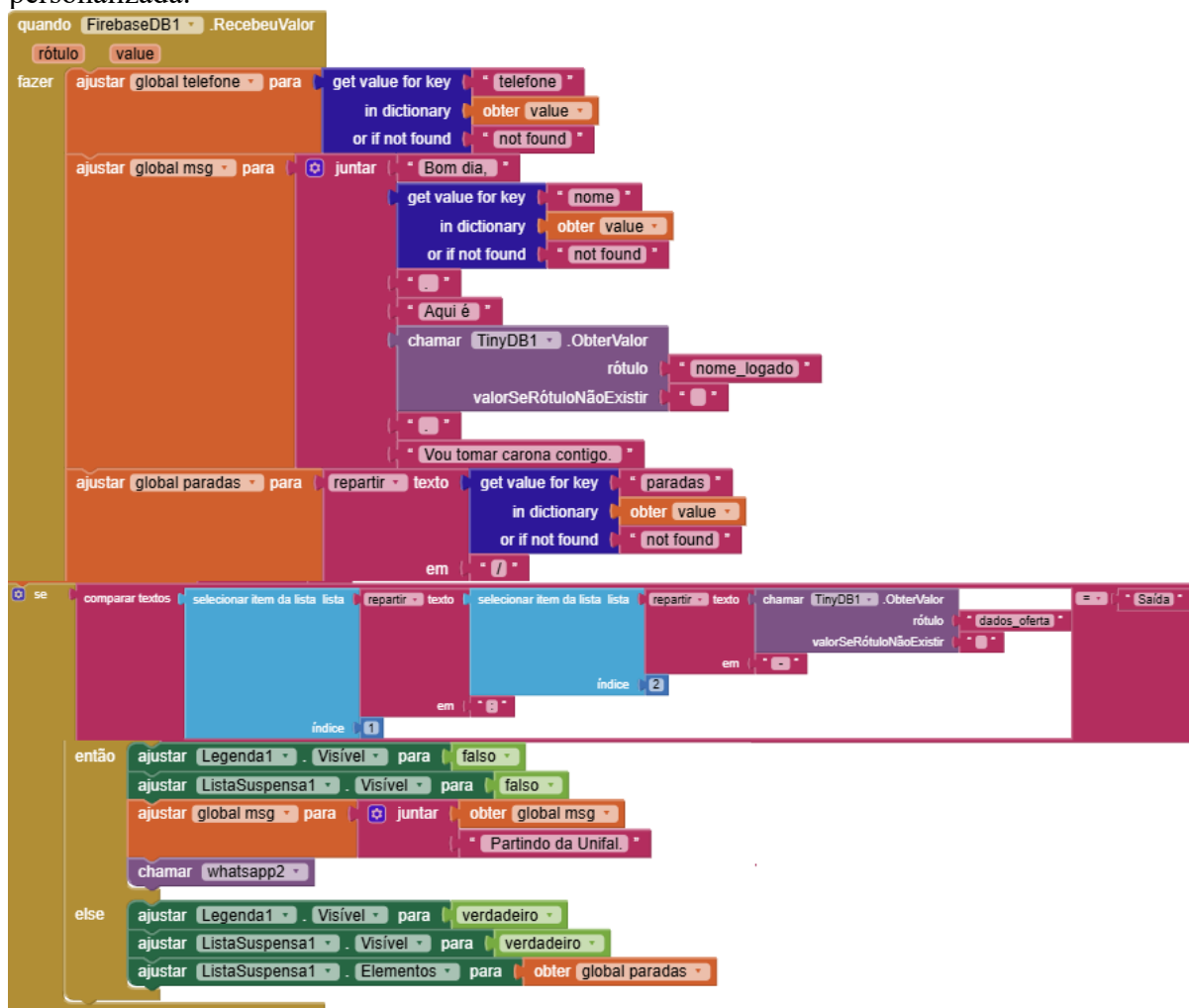


Recebimento de Dados do Firebase

Na Figura 31, apresenta-se o bloco de código responsável pelo processamento dos dados retornados do Firebase, geralmente relacionados às informações de um usuário ofertante de carona. Inicialmente, o sistema extrai, a partir do dicionário recebido, o número de telefone e o nome do motorista, os quais são armazenados nas variáveis globais `telefone` e `msg`, respectivamente. Em seguida, o conteúdo da variável `msg` é atualizado com uma mensagem personalizada contendo o nome do motorista, com o objetivo de facilitar a comunicação entre passageiro e condutor.

O sistema também verifica a existência do campo "paradas", e, caso ele esteja presente, seus valores são armazenados na variável "global paradas". Esse procedimento tem como finalidade permite que o passageiro escolha, entre diferentes opções, o ponto de embarque mais conveniente. Assim, quando o ponto de partida não é o próprio campus, a mensagem personalizada ao motorista é atualizada para incluir o ponto de parada selecionado.

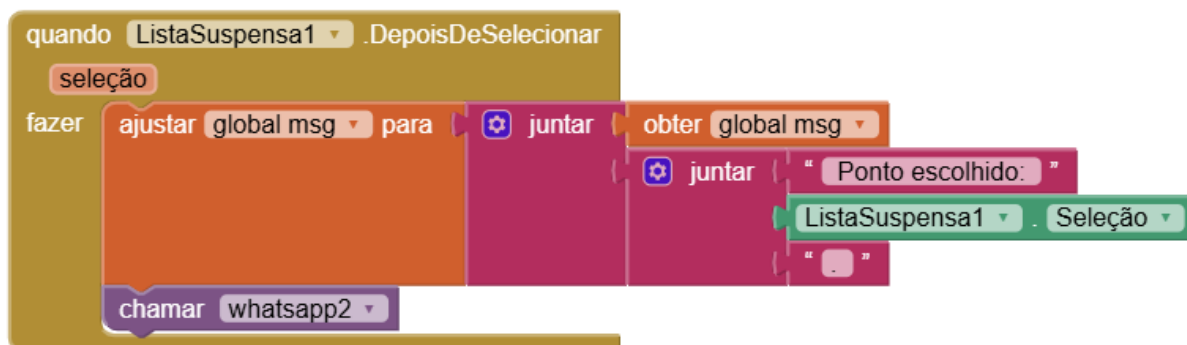
Figura 31 - Blocos que processam dados recebidos do Firebase para montar uma mensagem personalizada.



(Versão ampliada disponível no Apêndice A – Figura A1)

O bloco da Figura 32 é executado após o usuário selecionar um ponto de encontro na ListaSuspensa1. Ele atualiza a variável “global msg”, adicionando a informação do ponto escolhido, e em seguida aciona o componente whatsapp2 para enviar automaticamente essa mensagem ao motorista via WhatsApp, facilitando a comunicação entre passageiro e condutor da carona.

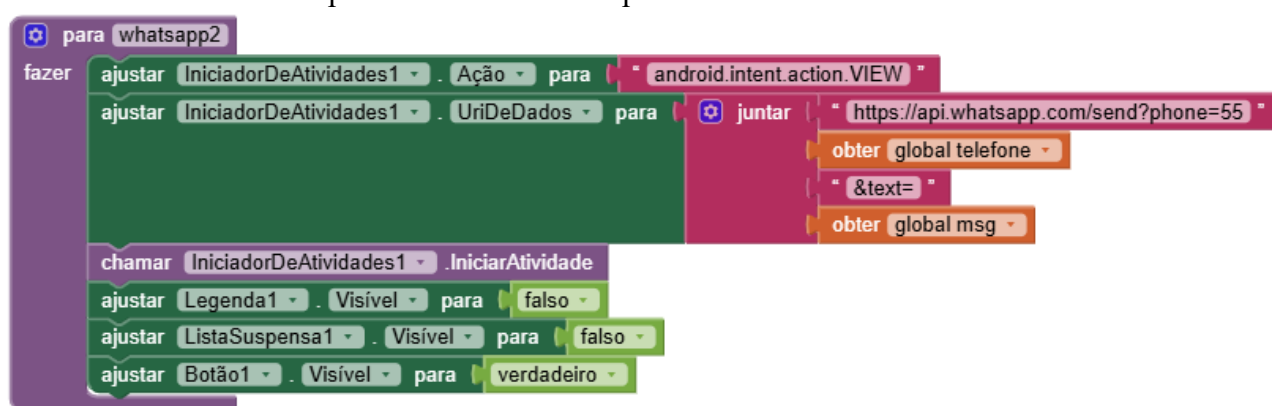
Figura 32 - Blocos que atualizam a mensagem global com o ponto selecionado em uma lista suspensa e acionam o envio da mensagem via WhatsApp.



Envio de Mensagem via WhatsApp

A chamada à função whatsapp2 tem como objetivo abrir uma conversa no WhatsApp com o motorista da carona, já contendo uma mensagem personalizada. Para isso, o sistema utiliza o componente IniciadorDeAtividade1, configurando uma URL no padrão da API do WhatsApp com o número de telefone (global_telefone) e a mensagem (global_msg). Ao executar essa ação, o aplicativo redireciona o usuário para o WhatsApp com o conteúdo pronto para envio. Após essa operação, a interface do aplicativo é atualizada, ocultando botões e campos relacionados à seleção da carona, sinalizando que o processo foi concluído com sucesso, vide Figura 33.

Figura 33 - Blocos que abrem uma conversa no WhatsApp com uma mensagem personalizada e ocultam ou exibem componentes da interface após o envio.



4.5 INTERFACE DE OFERTA DE CARONA (SCREEN3)

A Figura 34 apresenta a interface destinada ao ofertamento de carona por parte dos usuários do aplicativo. Nessa tela, o motorista pode registrar uma nova oferta de carona informando três dados principais: o horário da viagem, a quantidade de vagas disponíveis e o sentido do trajeto, selecionado a partir de uma lista suspensa (Spinner). Após o preenchimento dos campos, o botão "Cadastrar oferta" permite salvar essas informações no banco de dados, tornando a carona visível para os demais usuários interessados. A interface também conta com o botão "Voltar", que possibilita o retorno à tela anterior, garantindo uma navegação simples e intuitiva. Essa etapa é essencial para manter o sistema atualizado com as caronas ativas, permitindo o funcionamento dinâmico do aplicativo.

Figura 34 - Tela "Ofertamento de carona" do aplicativo, com campos para inserir horário, número de vagas, sentido da viagem e botões para cadastrar a oferta e voltar.

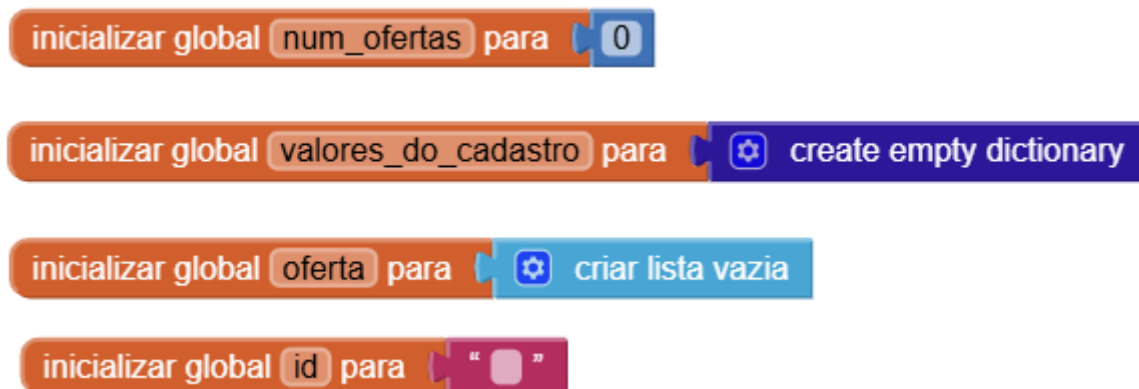
A imagem mostra a interface de uma tela de um aplicativo móvel. No topo, há uma barra de status com ícones de Wi-Fi, sinal de celular e bateria, e o horário 12:30. Abaixo, uma barra de título azul com o texto "Ofertamento de carona" e um ícone de menu. O formulário contém três campos de entrada: "Horário:" com dois campos adjacentes, "Vagas:" com um único campo, e "Sentido:" com uma lista suspensa que mostra "adicionar itens...". Abaixo dos campos, há um botão cinza com o texto "Cadastrar oferta". Logo abaixo dele, há um botão cinza com o texto "Voltar". O fundo da tela é branco, e a barra de navegação do Android está visível na base.

4.5.1 Implementação das Funcionalidades as Screen3

Inicialização de Variáveis Globais

- num_ofertas, ofertas, valores_do_cadastro, oferta e id: Variáveis globais são inicializadas para gerenciar o número de ofertas, a lista de ofertas disponíveis, e detalhes específicos do cadastro da oferta, como ilustrado na Figura 35.

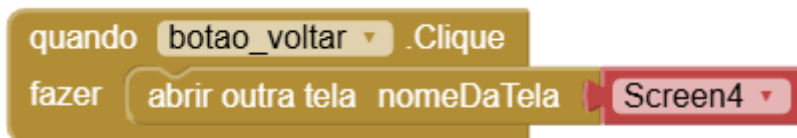
Figura 35 - Blocos de inicialização de variáveis globais usadas no controle e armazenamento de ofertas e dados de cadastro.



Bloco "Voltar" (Botão_voltar)

Quando o botão "Voltar" é clicado, o bloco executa o comando para fechar a tela atual e retornar à tela anterior (Screen 4), Figura 36 interrompendo o processo de ofertamento.

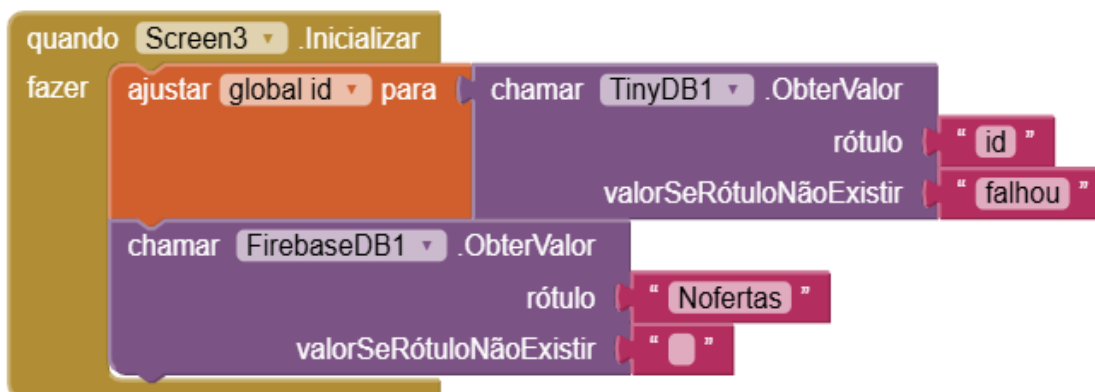
Figura 36 - Bloco que define a ação do botão "Voltar", redirecionando o usuário para a tela Screen4 ao ser clicado.



Inicialização da Tela e Carregamento de Dados (Screen3. Inicializar)

A Figura 37 representa o bloco executado na inicialização da Screen3, que corresponde à tela de gerenciamento das caronas ofertadas pelo usuário. Nesse momento, o sistema define a variável "global id" com base no valor armazenado no TinyDB1, utilizando o rótulo "id", que identifica o usuário logado. Caso esse valor não exista, a string "falhou" é atribuída como padrão. Em seguida, o sistema realiza uma chamada ao FirebaseDB1, solicitando os dados associados ao rótulo "Nofertas", que corresponde às caronas cadastradas no dia. Essa etapa garante que, ao abrir a tela, o aplicativo carregue corretamente as informações do usuário e suas ofertas ativas, preparando os dados para exibição ou edição conforme necessário.

Figura 37 - Blocos que inicializam a tela Screen3 recuperando o ID do usuário no TinyDB e os dados de ofertas no Firebase.

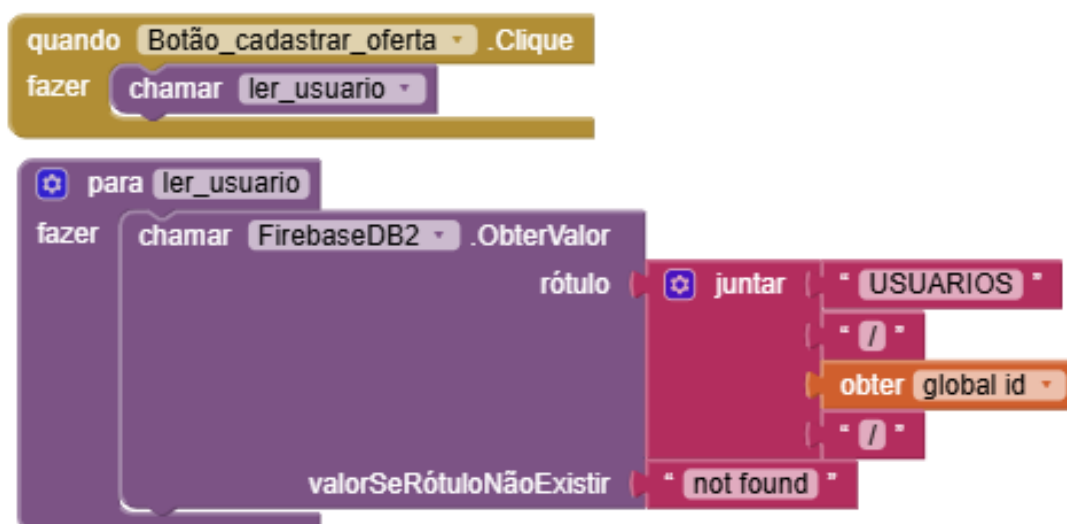


Botão “Cadastrar Oferta”

A Figura 38 apresenta o bloco de programação executado quando o usuário clica no botão “Cadastrar oferta”, indicando sua intenção de registrar uma nova carona no sistema. Nesse momento, o aplicativo chama a função ler_usuario, responsável por buscar no Firebase os dados do usuário atualmente logado.

Dentro dessa função, é utilizado o bloco FirebaseDB2.ObterValor para acessar o rótulo construído dinamicamente a partir da junção da palavra "USUARIOS" com o conteúdo da variável “global id”, que representa o identificador único do usuário. Esse rótulo corresponde ao caminho onde estão armazenadas as informações cadastrais do ofertante no Firebase. Caso o rótulo não exista, o valor retornado será "not found".

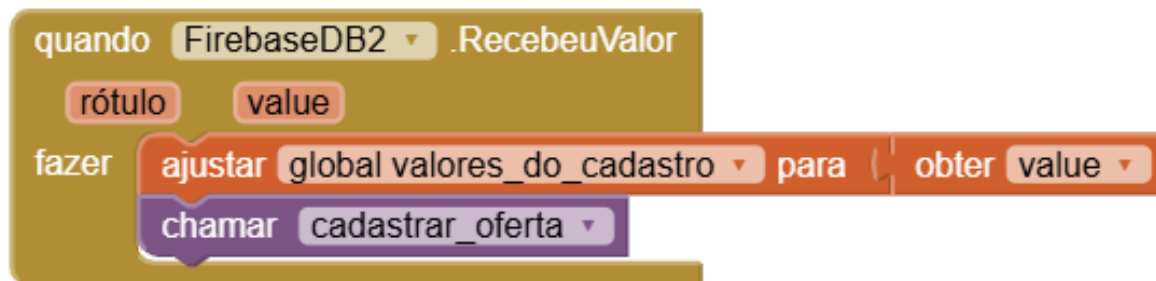
Figura 38 - Blocos que, ao clicar no botão "Cadastrar oferta", chamam a função para buscar os dados do usuário no Firebase utilizando o ID armazenado.



A Figura 39 apresenta o bloco executado quando o aplicativo recebe um valor do banco de dados Firebase, por meio do evento FirebaseDB2.RecebeuValor. Nessa etapa, o valor retornado é atribuído à variável global “valores_do_cadastro”, que armazena temporariamente as informações do usuário necessárias para registrar a nova carona. Em seguida, o sistema chama a função “cadastrar_oferta”, que utilizará os dados armazenados para concluir o processo de registro da carona no banco. Esse

bloco garante que o cadastro de ofertas só ocorra após a obtenção completa e correta dos dados do usuário, assegurando consistência nas informações gravadas no sistema.

Figura 39 - Blocos que, ao receberem dados do FirebaseDB2, armazenam os valores na variável global e chamam a função de cadastro de oferta.



Bloco de Preparação de Detalhes (para montar_oferta)

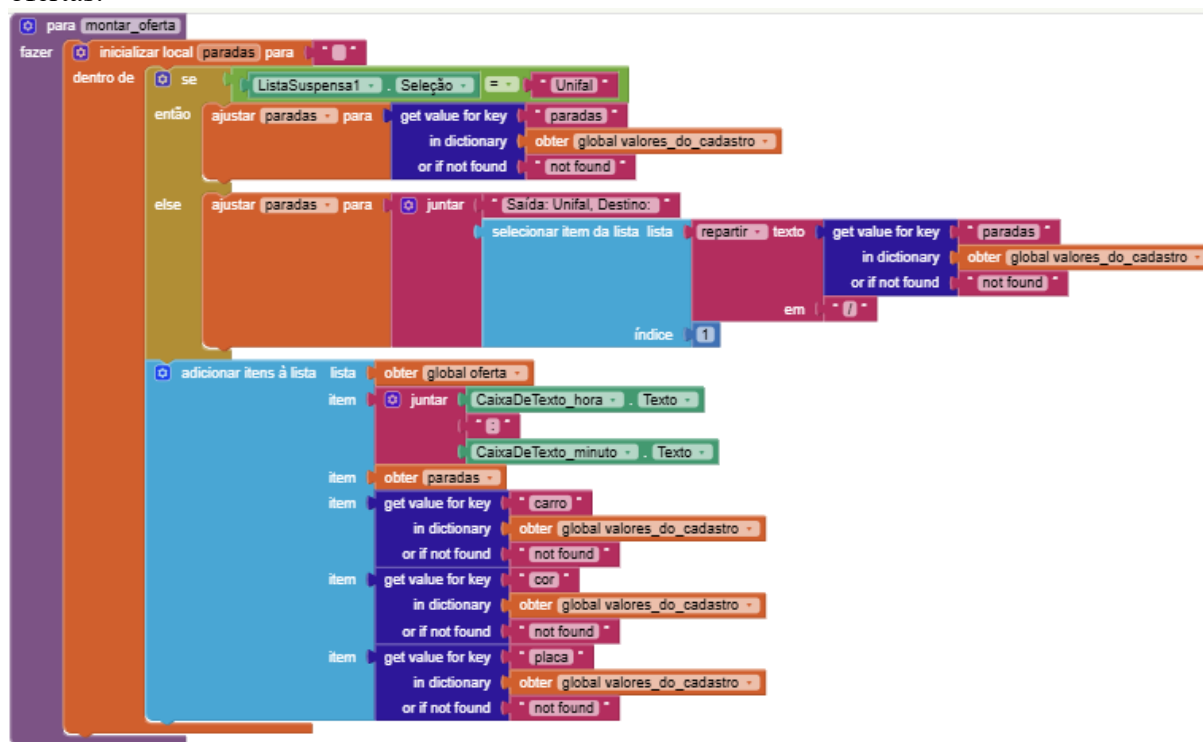
A Figura 40 apresenta o bloco da função `montar_oferta`, responsável por organizar e registrar os dados fornecidos pelo usuário para a criação de uma nova oferta de carona. Essa função coleta os dados inseridos nos campos da tela e os estrutura em um formato padronizado, armazenando as informações em uma lista global.

Inicialmente, o sistema verifica o valor selecionado no componente `ListaSuspensa1`. Caso a opção escolhida seja "Unifal", o campo paradas recebe os valores armazenados no dicionário `valores_do_cadastro` sob a chave "Unifal". Caso contrário, o sistema constrói a string juntando os dados com o texto "Saída: Unifal, Destino: ".

Em seguida, o sistema adiciona os dados da oferta à lista global `oferta`. Os itens adicionados incluem: o horário e lista de paradas, montada anteriormente; o modelo do carro, cor e placa, todos recuperados do dicionário `valores_do_cadastro`.

Essa organização permite que os dados sejam padronizados e utilizados posteriormente na exibição das ofertas ou no envio das informações para o banco de dados. A função garante que a carona seja montada com todas as informações essenciais, oferecendo clareza tanto para o passageiro quanto para o sistema.

Figura 40 - Blocos da função "montar_oferta" que compõem os dados de uma nova oferta com base nas informações inseridas pelo usuário e nos dados do cadastro, adicionando-os à lista de ofertas.



(Versão ampliada disponível no Apêndice A – Figuras A2)

Bloco para Gravar Ofertas (para gravar_ofertas)

A Figura 41 apresenta os blocos das funções cadastrar_oferta e gravar_oferta, responsáveis por finalizar o processo de registro de uma nova carona no aplicativo. Esse processo ocorre após o preenchimento dos campos da tela de ofertamento e envolve tanto a preparação quanto o envio dos dados ao banco de dados Firebase. A função cadastrar_oferta inicia chamando montar_oferta, que organiza os dados da carona em uma estrutura padronizada. Em seguida, o sistema atualiza a variável global num_ofertas, incrementando o total de ofertas registradas, e exibe um alerta ao usuário com a mensagem “Oferta cadastrada”.

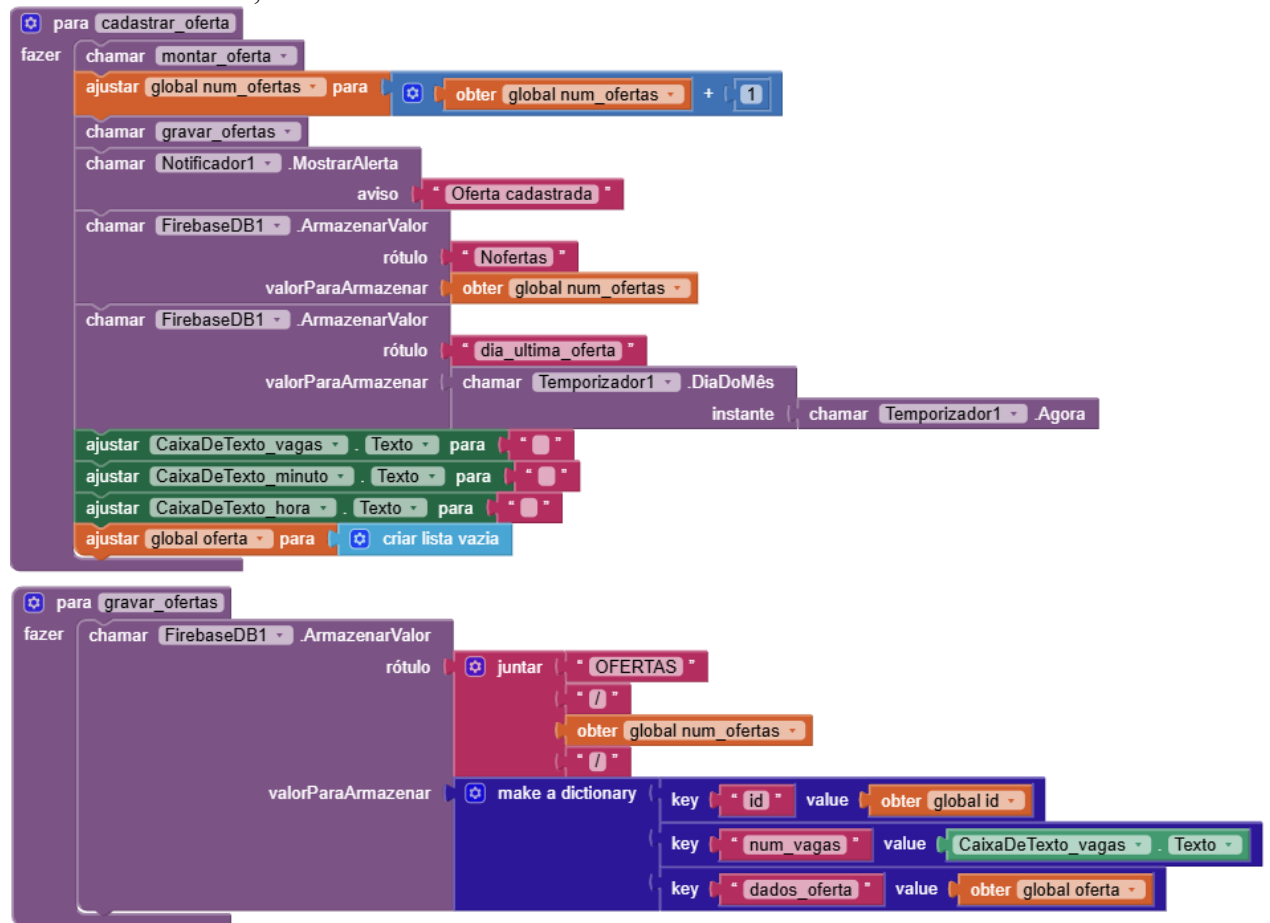
Logo após, a função gravar_oferta é chamada para salvar os dados da nova carona no Firebase. Ela armazena no rótulo "OFERTAS", usando como índice "Nofertas" um dicionário contendo: id do usuário (motorista), o número de vagas e os dados_oferta montados previamente.

Além disso, o número total de ofertas (global num_ofertas) também é salvo no rótulo "Nofertas" adicionando 1, e o dia do cadastro é registrado no rótulo "dia_ultima_oferta", utilizando a data atual obtida via o componente Temporizador1.

Por fim, os campos da interface relacionados à oferta (vagas, horário e dados da carona) são resetados para seus valores iniciais, preparando o aplicativo para um novo cadastro, caso necessário.

Esse bloco garante que cada oferta registrada seja devidamente estruturada, salva com um identificador único, e sincronizada com o banco de dados, mantendo a consistência do sistema.

Figura 41 - Blocos das funções "cadastrar_oferta" e "gravar_ofertas", responsáveis por montar os dados da carona, armazená-los no Firebase e atualizar o número total de ofertas.



4.6 JORNADA DO USUÁRIO



Nesta seção, apresenta-se o blueprint de serviço do aplicativo UNICAR, vide Figura 42 com foco na jornada do usuário, evidenciando as interações frontstage, processos de suporte (tecnologias) e as conexões entre interface e lógica de negócios. O diagrama a seguir sintetiza o fluxo completo desde o login até a oferta ou solicitação de carona, incluindo os serviços utilizados em cada etapa.

Figura 422 - UNICAR: UM APLICATIVO DE CARONAS COMPARTILHADAS PARA A UNIVERSIDADE FEDERAL DE ALFENAS

Blueprint de Serviço UNICAR: Jornada do Cliente

Linha de Visualização e ações do Cliente (Frontstage)						
Tempo: Instantâneo	Tempo: Instantâneo	Tempo: 0 - 3 min	Tempo: 0 - 1 min	Tempo: 0 - 1 min	Tempo: Variável	Tempo: Variável
Tela Inicial	Ação: Clica em "Registrar" ou "Entrar"	Caminho de Registro	Caminho de Login	Tempo: Instantâneo	Caminho "Oferecer"	Caminho "Solicitar"
Visualiza campos de E-mail e Senha, botões "Entrar" e "Registrar".	Ação: Clica em "Registrar" ou "Entrar" O cliente insere suas credenciais.	Ação: Preenche Nome, Telefone. Ação: Clica em "Cadastrar Carro" (Carro, Cor, Placa). Ação: Clica em "Adicionar Parada" (Pontos de Parada Usuais). Ação: Clica em "Concluir Registro".	O usuário preenche as credenciais e clica em "Entrar".	Visualiza caminhos "Oferecer", "Solicitar", "Encerrar".	O usuário preenche os detalhes da carona (horário, vagas) e clica em "Cadastrar oferta".	Ação: Clica em "Solicitar". Tela de Solicitação: Visualiza lista de ofertas. Ação: Seleciona uma oferta. Diálogo de Confirmação: "Confirma a escolha?" (Sim/Não). Se "Não": "Solicitação não confirmada". Se "Sim": Tela de Seleção de Ponto de Parada. Ação: Clica em botão de envio. Abertura do WhatsApp: Mensagem personalizada pré-preenchida.
Conexão Frontstage com Backstage (Lógica de Negócios)						
Tempo: Instantâneo	Tempo: Instantâneo	Tempo: 0 - 3 min	Tempo: 0 - 1 min	Tempo: Instantâneo	Tempo: Variável	Tempo: Variável
Tela Inicial	Autenticação Firebase	Armazenamento de Dados	Autenticação no Firebase	Tela de Funcionalidades	Armazenamento De Carona	Integração WhatsApp API WhatsApp
	Inicia o fluxo de autenticação ou registro.	Os dados do perfil (usuário, carro, paradas) são salvos de forma estruturada no banco de dados Firebase	O fluxo de autenticação é iniciado para validar o usuário.	Encaminha para a tela desejada	O sistema processa o registro da nova oferta, tornando-a visível para outros usuários na plataforma.	O app gera e abre uma mensagem personalizada no WhatsApp para conectar o passageiro e o motorista.
Linha de Ações do Backstage						
Tempo: Instantâneo	Tempo: Instantâneo	Tempo: 0 - 3 min	Tempo: 0 - 1 min	Tempo: Instantâneo	Tempo: Variável	Tempo: Variável
Tela Inicial	Ação: Clica em "Registrar" ou "Entrar"	Caminho de Registro	Caminho de Login	Tela de Funcionalidades	Caminho "Oferecer"	Caminho "Solicitar"
	Firebase Authentication	Armazenamento no Firebase em forma de dicionário (JSON)	O sistema faz a chamada ao Firebase Authentication, recebe e decodifica a resposta JSON para obter o token de identificação (idToken) e prosseguir	Abrir outra tela	O sistema busca os dados cadastrais do motorista logado, monta a estrutura de dados (dicionário) da oferta e a armazena no Firebase Realtime Database sob a tag "OFERTAS", atualizando o contador de ofertas totais	Busca, processa, formata a lista de ofertas, atualiza vagas e busca dados do ofertante para montar a mensagem.
Processos de Suporte (Tecnologias)						
Plataforma MIT App Inventor: Sustenta a interface e a lógica inicial. Firebase Realtime Database: O app já faz uma chamada inicial para verificar a data da última oferta.	Firebase Authentication: É o serviço de destino para a validação das credenciais.	Firebase Authentication: Para a criação da nova conta de usuário.	Firebase Authentication: Para validar o e-mail e a senha fornecidos.	Plataforma MIT App Inventor: Responsável pela navegação entre as telas (Screen4 -> Screen2 ou Screen3).	Firebase Realtime Database: Para buscar os dados cadastrais do motorista e, em seguida, gravar a nova oferta de carona, atualizando o contador de ofertas.	Firebase Realtime Database: Para ler a lista de ofertas disponíveis, e depois para atualizar a oferta escolhida (diminuindo o número de vagas) e buscar os dados do motorista.

6. conclusão O desenvolvimento do aplicativo UNICAR, voltado para a comunidade acadêmica da UNIFAL-MG, mostrou-se uma alternativa promissora frente aos desafios de mobilidade urbana enfrentados por estudantes, professores e técnicos-administrativos. A proposta demonstrou ser tecnicamente viável, com a construção de um protótipo funcional capaz de realizar autenticação de usuários, cadastro e solicitação de caronas, com suporte de banco de dados em nuvem e interface intuitiva.

A adoção de tecnologias acessíveis como o MIT App Inventor e o Firebase permitiu viabilizar a criação do sistema mesmo com recursos limitados, reforçando a importância do uso educacional dessas plataformas na formação tecnológica. Os testes realizados pelos próprios desenvolvedores indicam que o sistema está apto para ser utilizado em pequena escala, com potencial para expansão futura.

Embora o projeto não tenha alcançado a etapa de validação externa com usuários reais, a estrutura lógica e os fluxos de dados desenvolvidos consolidam uma base sólida para possíveis evoluções, representando uma contribuição relevante tanto na perspectiva técnica quanto didática.

7. TRABALHOS FUTUROS

A continuidade deste projeto poderá seguir por diversas frentes:

- Realizar **testes com grupos piloto** de usuários, de forma controlada, para avaliar usabilidade, confiabilidade e impacto na mobilidade;
- Implementar um **histórico de uso** e sistema de avaliação de usuários, permitindo maior segurança e rastreabilidade;
- Criar **mapas de jornada do usuário** e **blueprints de serviço**, documentando visualmente os fluxos e pontos de contato para melhoria da experiência;
- Avaliar a **viabilidade institucional**, com possibilidade de parceria com a universidade para lançamento oficial;
- Ampliar funcionalidades, como **geolocalização**, **rotas inteligentes**, **notificações push** e integração com sistemas de transporte público;
- Realizar simulações e testes de desempenho para avaliar **escalabilidade** da infraestrutura;
- Explorar modelos de **sustentabilidade financeira** e **modelos de negócio**, caso haja interesse em expandir para outras instituições.

Outro aspecto relevante a ser considerado em trabalhos futuros refere-se à escalabilidade da aplicação. Embora o protótipo tenha sido desenvolvido com foco no campus Poços de Caldas da UNIFAL-MG, sua estrutura pode ser adaptada para atender aos demais campus da universidade. No entanto, para alcançar uma escala institucional mais ampla, seria necessário migrar para um plano pago do Firebase, que oferece maior capacidade de armazenamento e suporte para até 200 mil conexões simultâneas. Nessa perspectiva, a manutenção do sistema exigiria o envolvimento institucional direto — como a adoção do aplicativo por parte da universidade — ou a gestão por entidades como o DCE, mediante a cobrança simbólica de assinatura dos usuários para viabilizar a continuidade operacional da plataforma.

8. CONSIDERAÇÕES FINAIS

O UNICAR representa mais do que uma proposta de aplicativo: é uma resposta técnica a um problema cotidiano vivenciado por muitos estudantes do ensino superior. Seu desenvolvimento evidencia como ferramentas de baixo custo e lógica de programação visual podem ser aplicadas para propor soluções reais dentro da universidade.

O caráter didático do projeto se destaca ao demonstrar, de forma acessível, os fundamentos da construção de um sistema funcional, o que pode servir como base para estudantes interessados em programação e inovação social. Mais do que resolver a questão da mobilidade, a proposta estimula práticas colaborativas, reforça valores de sustentabilidade e promove a autonomia tecnológica dentro do ambiente acadêmico.

Com os devidos aprimoramentos e envolvimento institucional, a iniciativa poderá evoluir para além de um protótipo, consolidando-se como uma ferramenta útil para a comunidade acadêmica e, futuramente, como referência replicável em outras universidades do país.

9- REFERÊNCIAS

COHEN, M.; SUNDARARAJAN, A. Self-Regulation and Innovation in the Peer-to-Peer Sharing Economy. *University of Chicago Law Review Online*, v. 82, n. 1, p. 116–133, 2015. Disponível em: https://chicagounbound.uchicago.edu/uclrev_online/vol82/iss1/8/. Acesso em: 3 jun. 2025.

CONTEXT NEUTRAL. Top Best Practices for Firebase Authentication in Node.js: Secure and Efficient Techniques. 17 fev. 2024. Disponível em: <https://www.contextneutral.com>. Acesso em: 3 jun. 2025.

DAVIS, R. T. MIT Launches Social Network for Ridesharing. *MIT News*, 12 out. 2011. Disponível em: <https://news.mit.edu/2011/mit-launches-social-network-ridesharing-1012>. Acesso em: 3 jun. 2025.

ERT, E.; FLEISCHER, A.; MAGEN, N. Trust and Reputation in the Sharing Economy: The Role of Personal Photos in Airbnb. *Tourism Management*, v. 55, p. 62–73, 2016. Disponível em: <https://doi.org/10.1016/j.tourman.2016.01.013>. Acesso em: 3 jun. 2025.

EXAME. Universitários lançam aplicativo para compartilhar caronas. *Exame*, 2016. Disponível em: <https://exame.com/noticias-sobre/caronaê/>. Acesso em: 3 jun. 2025.

FURUHATA, M. et al. Ridesharing: The State-of-the-Art and Future Directions. *Transportation Research Part B: Methodological*, v. 57, p. 28-46, 2013. Disponível em: <https://doi.org/10.1016/j.trb.2013.08.012>. Acesso em: 23 dez. 2024.

FUVESTIBULAR. Ex-alunos da USP criam aplicativo de carona para universitários. 2015. Disponível em: <https://fuvestibular.com.br/noticia/ex-alunos-da-usp-criam-aplicativo-de-carona-para-universitarios/>. Acesso em: 3 jun. 2025.

GOOGLE. Firebase Authentication. Disponível em: <https://firebase.google.com/products/auth>. Acesso em: 3 jun. 2025a.

GOOGLE. Firebase Realtime Database. Disponível em: <https://firebase.google.com/docs/database>. Acesso em: 3 jun. 2025b.

HAMARI, J.; SJÖKLINT, M.; UKKONEN, A. The Sharing Economy: Why People Participate in Collaborative Consumption. *Journal of the Association for Information Science and Technology*, v. 67, n. 9, p. 2047-2059, 2016. Disponível em: <https://doi.org/10.1002/asi.23552>. Acesso em: 23 dez. 2024.

HARIHARAN, R.; MADDEN, S. TinyDB: An Acquisitional Query Processing System for Sensor Networks. *ACM Transactions on Database Systems*, v. 28, n. 1, p. 3-44, 2003. Disponível em: <https://doi.org/10.1145/637593.637595>. Acesso em: 23 dez. 2024.

INTERNATIONAL TRANSPORT FORUM. The Health Impacts of Low-Carbon Transport in Cities: Policy Brief. Paris: OECD Publishing, 2024. Disponível em: <https://www.itf-oecd.org/sites/default/files/docs/health-impacts-low-carbon-transport-cities.pdf>. Acesso em: 3 jun. 2025.

JONES, R. VCU Launches Social Network for Ridesharing. *VCU News*, 5 out. 2010. Disponível em: https://www.news.vcu.edu/article/VCU_Launches_Social_Network_for_Ridesharing. Acesso em: 3 jun. 2025.

MIT APP INVENTOR COMMUNITY. Firebase DB with App Inventor Tutorial. Disponível em: <https://community.appinventor.mit.edu/t/firebase-db-with-app-inventor-tutorial/43872>. Acesso em: 3 jun. 2025.

PATEL, H. Simple Database Management in MIT App Inventor 2 with TinyDB, TinyWebDB, and Fusion Tables. *MIT App Inventor*, 2016. Disponível em: <https://appinventor.mit.edu/explore>. Acesso em: 23 dez. 2024.

PERKINS, M. Ride-Sharing Program Comes to USU. *USU Today*, 22 jan. 2015. Disponível em: <https://www.usu.edu/today/?id=57702>. Acesso em: 3 jun. 2025.

SHAHEEN, S. et al. Shared Mobility: Definitions, Industry Developments, and Early Understanding. *Transportation Sustainability Research Center*, University of California, Berkeley, 2015. Disponível em: <https://tsrc.berkeley.edu/publications/shared-mobility-definitions-industry-developments-and-early-understanding>. Acesso em: 3 jun. 2025.

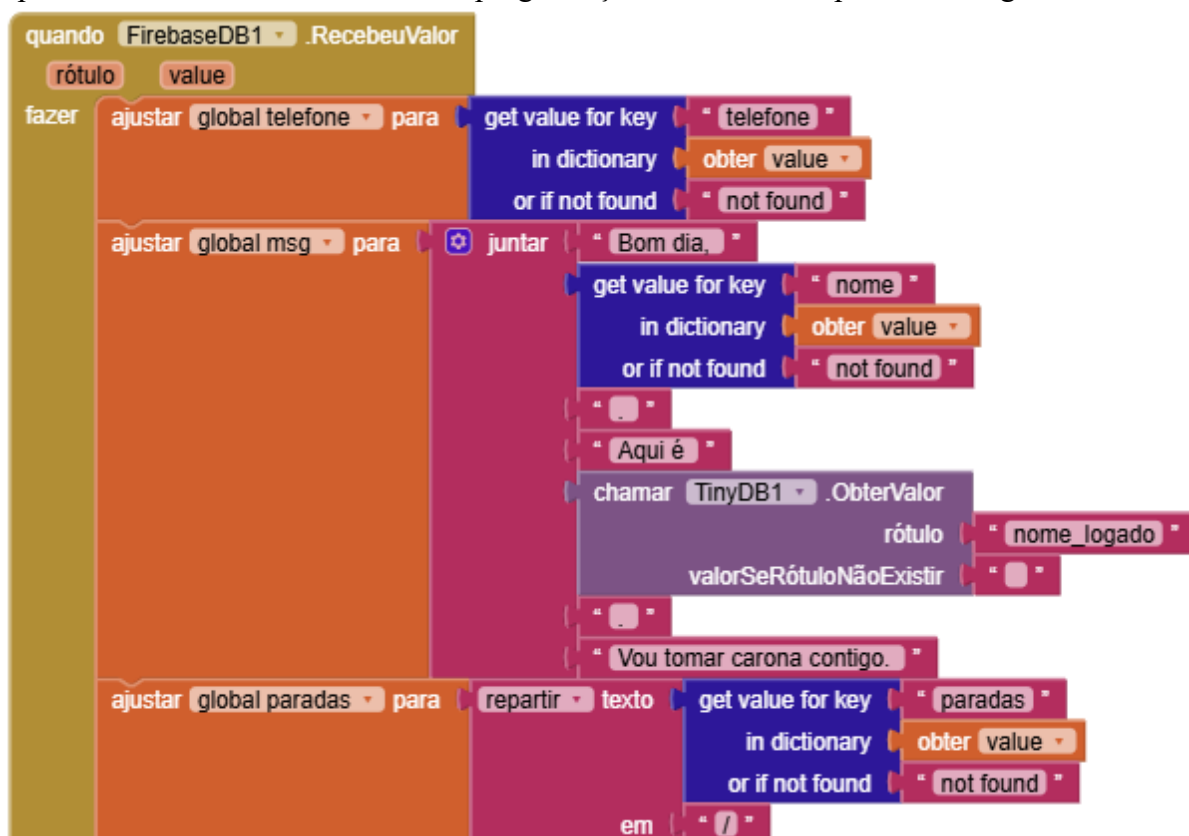
SHAHEEN, S.; COHEN, A.; ZOHDY, I. Shared Mobility: Current Practices and Guiding Principles. USDOT Federal Highway Administration, U.S. Department of Transportation, 2016. Disponível em: <https://ops.fhwa.dot.gov/publications/fhwahop16022/index.htm>. Acesso em: 23 dez. 2024.

TZIMOS, D. N. Carpooling Platforms as Smart City Projects: A Bibliometric Analysis and Systematic Literature Review. *Sustainability*, v. 13, n. 19, p. 10680, 2021. Disponível em: <https://doi.org/10.3390/su131910680>. Acesso em: 23 dez. 2024.

10- APÊNDICE – FIGURAS AMPLIADAS DO APLICATIVO UNICAR

Este apêndice apresenta as versões ampliadas das Figuras 31 e 40, com o objetivo de facilitar a visualização dos blocos de programação utilizados no desenvolvimento do aplicativo UNICAR. As ampliações visam permitir a análise mais clara dos detalhes contidos nos componentes visuais programados no MIT App Inventor.

Apêndice 1 – Parte 1/2 dos Blocos de programação detalhados ampliados da Figura 31.



Apêndice 2 – Parte2/2 dos Blocos de programação detalhados ampliados da Figura 31.

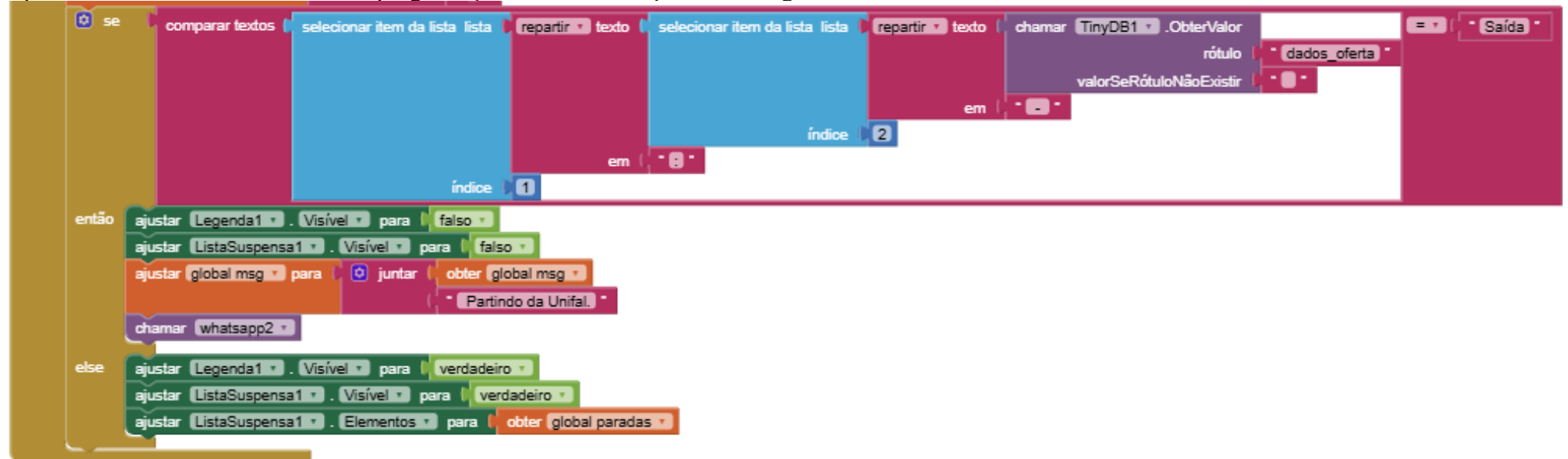


Figura A2 – Blocos de programação detalhados da Figura 40.

Apêndice 3 – Parte 1/2 dos Blocos de programação detalhados ampliados da Figura 40.



Apêndice 4 - Parte 2/2 dos Blocos de programação detalhados ampliados da Figura 40.

